



Universal Composable Password Authenticated Key Exchange for the Post-Quantum World

You Lyu^{1,2}, Shengli Liu^{1,2}, and Shuai Han^{2,3}

¹ Department of Computer Science and Engineering, Shanghai Jiao Tong University,
Shanghai 200240, China

{[vergil](mailto:vergil@sjtu.edu.cn),[slliu](mailto:slliu@sjtu.edu.cn)}@sjtu.edu.cn

² State Key Laboratory of Cryptology, P.O. Box 5159, Beijing 100878, China

³ School of Cyber Science and Engineering, Shanghai Jiao Tong University,
Shanghai 200240, China
dalen17@sjtu.edu.cn

Abstract. In this paper, we construct the *first* password authenticated key exchange (PAKE) scheme from isogenies with Universal Composable (UC) security in the random oracle model (ROM). We also construct the *first* two PAKE schemes with UC security in the quantum random oracle model (QROM), one is based on the learning with error (LWE) assumption, and the other is based on the group-action decisional Diffie-Hellman (GA-DDH) assumption in the isogeny setting.

To obtain our UC-secure PAKE scheme in ROM, we propose a generic construction of PAKE from basic lossy public key encryption (LPKE) and CCA-secure PKE. We also introduce a new variant of LPKE, named extractable LPKE (eLPKE). By replacing the basic LPKE with eLPKE, our generic construction of PAKE achieves UC security in QROM. The LPKE and eLPKE have instantiations not only from LWE but also from GA-DDH, which admit four specific PAKE schemes with UC security in ROM or QROM, based on LWE or GA-DDH.

1 Introduction

Password Authenticated Key Exchange (PAKE) enables two parties (say, a client and a server) who possess a low-entropy password pw to securely establish session keys over public networks. These session keys subsequently facilitate the establishment of secure communication channels. Unlike authenticated key exchange (AKE), which necessitates a Public Key Infrastructure (PKI) to verify the authenticity of public keys, PAKE runs with easily memorable passwords and offers enhanced convenience for deployments and applications.

Security Notions for PAKE: IND vs. UC. There are two primary security notions for PAKE, the game-based security in the Indistinguishability model (IND security) [9] and the simulation-based security under the Universally Composable framework (UC security) [16]. As shown in [16], the UC security in the UC framework implies the IND security. In contrast to the IND model which

assumes passwords uniformly distributed over a set, the UC framework permits arbitrary correlations and distributions for passwords and guarantees security amidst composition with arbitrary protocols and hence is a better security model.

PAKE with Post-Quantum Security. There are quite a few IND-secure PAKE schemes constructed from post-quantum assumptions, including lattice-based [11, 19, 23, 24, 34] and isogeny-based [3] ones.

As for UC security, there exists a generic construction for PAKE from Oblivious Transfer (OT) [15]. However, achieving UC security for PAKE requires the underlying OT protocol have adaptive UC security even in absence of authenticated channels. As far as we know, up to now such OT only has instantiations from number-theoretic assumptions like CDH or Factoring, so no instantiation from post-quantum assumptions via the OT approach is known for PAKE. There also exist frameworks of constructing PAKE from Hash Proof System (HPS) (e.g., [2]). To apply the HPS framework, one needs an HPS for language consisting of pairs of messages and commitments/ciphertexts (m, c) , but as far as we know there is no suitable HPS from lattice/isogenies serving for UC-secure PAKE. A feasible approach to PAKE from post-quantum assumptions is making use of Encrypted Key Exchange [10] (EKE) and resorting to the Ideal Cipher Model (ICM) to achieve UC security. This EKE approach results in two UC-secure PAKE schemes [8, 31] both of which are lattice-based. However, the ICM has two limitations and this leads to two questions.

- It is unclear how to instantiate ideal cipher from isogenies, as highlighted in [7]. Hence isogeny-based PAKE with UC security is now missing.

Q1: Can we construct UC-secure PAKE from isogeny-based assumptions?

- ICM does not consider quantum access from adversaries and it is unclear how to exploit quantum ICM (QICM) to achieve security against quantum algorithms. But quantum random oracle model (QROM) [13] takes into account quantum-access adversaries, and is better understood than QICM [22].

Q2: Can we construct PAKE protocols with UC security in QROM ?

Our Contribution. We answer the above two questions in this paper with the following two folds of contributions.

1. We propose a generic construction for UC-secure PAKE in the random oracle model (ROM) from two building blocks, a basic Lossy Public Key Encryption (LPKE) and a CCA-secure PKE.
 - The pivot of our generic construction is Lossy Public Key Encryption (LPKE). We identify properties for basic LPKE so that its integration with hashing function makes UC security possible for PAKE in ROM.
 - The instantiations of the basic LPKE and CCA-secure PKE yield two UC-secure PAKE schemes in ROM, one is based on the LWE assumption and the other based on the GA-DDH assumption, which leads to the *first* PAKE scheme with UC security from isogenies.
2. We upgrade UC security from ROM to QROM for our generic PAKE construction, by replacing the “basic LPKE + Hash” with an extractable LPKE.

- We define extractable LPKE by equipping it with an extracting algorithm and identify its properties to make UC security possible for PAKE in QROM. We also present a generic construction for extractable LPKE.
- The instantiations of extractable LPKE lead to the *first* two UC-secure PAKE schemes in QROM, one is based on the LWE assumption and the other based on the GA-DDH assumption in the isogeny setting.

Technique Overview. The design principle of our PAKE is to make the underlying PKE associating with passwords. To this end, we introduce a labeled lossy public key encryption LPKE, where the passwords pw are used to derive labels $b := H(pw)$ for LPKE via hash function. The labels are used to generate public keys pk and secret keys sk , i.e., $(pk, sk) \leftarrow \text{LPKE.LKeyGen}(b = H(pw))$, and the encryptions also involve labels, i.e., $c \leftarrow \text{LPKE.LEnc}(pk, b = H(pw), m)$. Moreover, LPKE has an algorithm named $\text{IsLossy}(td, pk, b)$, which can use the trapdoor td to tell whether b is a lossy label for pk . If the output is 0, then b must be a normal label for pk , and hence LPKE works in normal mode, which has correctness and CPA security. Otherwise b must be a lossy label, and hence LPKE works in lossy mode. When applying FO-transformation to LPKE, it has CCA security in normal mode but has pseudo-random ciphertexts in lossy mode.

Together with another CCA-secure PKE scheme, we can design a 3-round PAKE scheme which is high-level shown in Fig. 2.

If client C and server S share the same password pw , then the label used to generate pk/sk and that used in the encryption are consistent. Accordingly, LPKE works in *normal* mode and the successful decryption of c guarantees C and S share $m = r|\sigma|k$. The ciphertext $C = \text{PKE.CEnc}(cpk, pw|pk|c; r)$ can be considered as a proof of server S 's knowledge of pw and the authentication of transcript $pk|c$, so it plays the role of authenticating S . Moreover, client C 's knowledge of σ from the decryption of c proves that C shares the same password with S , and hence the third message σ is able to authenticate C .

UC SECURITY IN ROM. To prove the UC security for our PAKE, we have to construct a simulator Sim , which has no knowledge of password pw but can simulate all the interaction transcripts for both passive attacks and active attacks. The simulations must be indistinguishable to those in the real experiment for an environment, which uses pw to control C and S , and manipulates adversaries $\mathcal{A}$ to

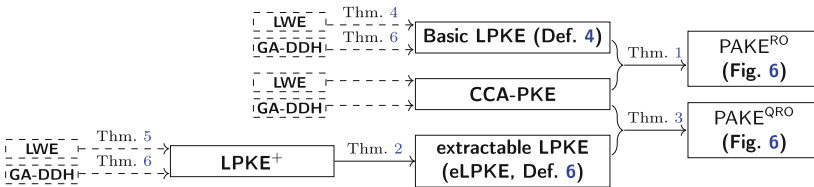


Fig. 1. Schematic overview of our PAKE constructions, where solid arrows “ $\longrightarrow$ ” indicate generic constructions and dashed arrows “ $\dashrightarrow$ ” indicate instantiations.

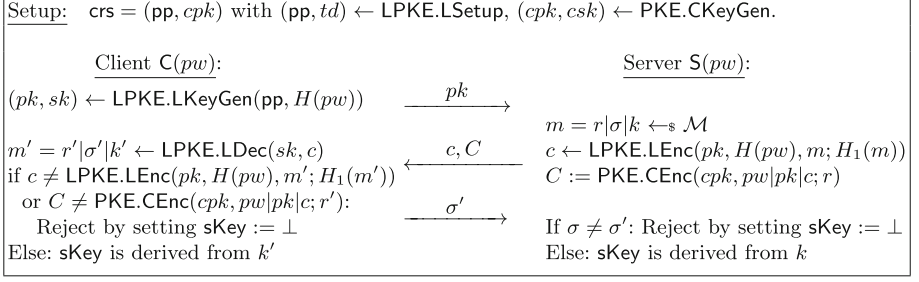


Fig. 2. Our PAKE from LPKE and PKE

interfere the interactions between C and S . Now we consider how the simulations are implemented according to the type of attacks, passive or active.

Case I: passive attacks. By requiring *pseudo-randomness of public key* for LPKE, simulator Sim can send a random pk as the simulation for the first-round message. The random pk is independent of the random label $H(pw)$ (due to random oracle), so $H(pw)$ is hardly the correct label generating pk . Then $c = \text{LPKE.LEnc}(pk, H(pw), m; H_1(m))$ works in lossy mode and c is pseudo-random due to *pseudo-randomness of ciphertext under lossy labels* of LPKE. Therefore, the simulation of c in the second-round message can be accomplished with a uniformly chosen one. As a result, $m = r|\sigma|k$ are random and independent of c . Then the *CCA security* of PKE implies $C \leftarrow \text{PKE.CEnc}(\text{cpk}, pw|pk|c; r) \approx_c C \leftarrow \text{PKE.CEnc}(\text{cpk}, 0; r)$. So ciphertext C in the second-round message can be simulated by $C \leftarrow_s \text{PKE.CEnc}(\text{cpk}, 0)$. Furthermore, the third-round message σ' can simply be simulated by setting $\sigma' := \sigma$ to keep consistence to the second-round message.

As for active attacks, a critical problem for Sim is that without any knowledge of pw , how to determine whether $\mathcal{A}$ implements attacks with a correct guess of pw . To solve this problem, we resort to the trapdoor td of LPKE, the algorithm IsLossy , random oracle H , and also the secret key of PKE.

Sim will generate the crs of PAKE and hold the trapdoor td of LPKE and the secret key csk of PKE to extract the possible passwords used in active attacks.

- **Password extraction from $\tilde{pk}$.** For a first-round message $\tilde{pk}$ from $\mathcal{A}$'s active attack, if $\tilde{pk}$ is associated with some password pw' , then $H(pw')$ must have been queried by $\mathcal{A}$. Sim just searches all the hash queries and replies under random oracle H to find $(pw', b = H(pw'))$ s.t. $\text{IsLossy}(td, \tilde{pk}, H(pw')) = 0$. If $\text{IsLossy}(td, \tilde{pk}, H(pw')) = 0$ then $H(pw')$ must be a normal label used to generate pk . Meanwhile, random oracle H makes sure the uniqueness of such password. In this way, the password pw' used by $\mathcal{A}$ for generating $\tilde{pk}$, if any, is successfully extracted by Sim .
- **Password extraction from $(\tilde{c}, \tilde{C})$.** For a second-round message $(\tilde{c}, \tilde{C})$ from $\mathcal{A}$'s active attack, Sim will use the secret key csk (w.r.t. the system public key

cpk) to decrypt $\tilde{C}$ to extract the encrypted password pw' . The correctness of PKE makes sure that the password pw' used by $\mathcal{A}$ for generating $(\tilde{c}, \tilde{C})$, if any, is successfully extracted by Sim.

Case II: active attacks. With help of the above extractions of password pw' , Sim can submit $\text{Testpw}(pw')$ to the ideal functionality of $\mathcal{F}_{\text{pake}}$ to decide whether pw' is the correct one. If pw' is correct, then Sim uses the correct pw' to simulate the protocol interactions, just like the real case. If the extracted password pw' is not correct, the simulations are implemented as follows.

- For a first-round message $\tilde{pk}$ from $\mathcal{A}$'s active attack, if the extracted pw' is not correct, then the random label $H(pw)$ must be lossy for $\tilde{pk}$. The simulator can simulate the second-round message (c, C) just like the case of passive attacks.
- For a second-round message $(\tilde{c}, \tilde{C})$ from $\mathcal{A}$'s active attack,
 - if the extracted pw' is not correct, the simulator just aborts the protocol. In the real case, the correctness of PKE also leads to abort due to an incorrect password.
 - if the extracted pw' is correct, recall that in this case, pk is a random one (simulated in the passive attack), the simulator has no knowledge of sk and cannot decrypt $\tilde{c}$ with LPKE.LDec . To solve this problem, we resort to random oracle $H_1(m')$ to extract the message $m' = r'|\sigma'|k'$ such that $\tilde{c} = \text{LPKE.LEnc}(pk, H(pw), m'; H_1(m'))$, if it exists.

UC SECURITY IN QROM. Note that random oracle H plays an important role in simulator's password extracting from $\tilde{pk}$. However, H fails in QROM, because the simulating technique of storing and searching queries & hash values in a list does not apply to QRO due to the no-cloning principle.

To achieve UC security in the QROM, our solution is discarding hash function H and directly using passwords pw as labels in the generation of $\tilde{pk}$ and the encryption of m . More importantly, we augment an $\text{Extract}(td, \tilde{pk})$ algorithm to LPKE which can use trapdoor td to extract password pw' from $\tilde{pk}$ directly, free of hash. In this way, LPKE is upgraded to an extractable one, namely eLPKE. The generic construction of PAKE remains the same except that we replace LPKE with eLPKE. The simulator can use $\text{Extract}(td, \tilde{pk})$ to extract pw' and the rest of simulations are almost the same as that in ROM.

We can construct extractable LPKE (eLPKE) from LPKE as follows. Parse the label bit-wise $pw = (pw_1, \dots, pw_\lambda)$. Each bit $pw_i \in \{0, 1\}$ corresponds to two public random tags v_i^0, v_i^1 . Then λ invocations of $(pk_i, sk_i) \leftarrow \text{LPKE.LKeyGen}(v_i^{pw_i})$ result in public key $\mathbf{pk} := (pk_1, \dots, pk_\lambda)$ and secret key $\mathbf{sk} := (sk_1, \dots, sk_\lambda)$. In this way, the password pw can be extracted bit-wisely via testing $\text{IsLossy}(td, pk_i, v_i^0) \stackrel{?}{=} 0$ and $\text{IsLossy}(td, pk_i, v_i^1) \stackrel{?}{=} 0$. For encryption, the plaintext m is divided into λ shares $m_1, \dots, m_\lambda$ such that $m = m_1 \oplus \dots \oplus m_\lambda$ via (λ, λ) -secret sharing. The ciphertext contains sub-ciphertexts $\{c_i := \text{LPKE.LEnc}(pk_i, m_i)\}_{i \in [\lambda]}$.

There remains a subtlety to be addressed. To justify that the real pk can be replaced by a random one, the security reduction must ensure that the

Table 1. Comparison of PAKE schemes from post-quantum assumptions. In [3, 11], a simultaneous flow of two round-messages is counted as 1-round while we count it as 2.

Scheme	Rounds	Security	Model	Assumption	Mutual Authentication	CRS
[11, 24]	2	IND	Standard	LWE	No	Yes
[34]	2	IND	RO	LWE	No	Yes
[23]	3	IND	Standard	LWE	Yes	Yes
[19]	3	IND	RO	RLWE	Yes	No
[3]	2	IND	RO	SqInv-GA-StCDH	No	Yes
[15] + [5]	3	IND	Standard	GA-DDH	Yes	Yes
[15] + [28]	3	IND	Standard	LWE	Yes	Yes
[8, 31]	2	UC	Ideal Cipher	MLWE	No	No
PAKE _{lwe} ^{RO}	3	UC	RO	LWE	Yes	Yes
PAKE _{ga} ^{RO}	3	UC	RO	GA-DDH	Yes	Yes
PAKE _{lwe} ^{QRO}	3	UC	QRO	LWE	Yes	Yes
PAKE _{ga} ^{QRO}	3	UC	QRO	GA-DDH	Yes	Yes

secret key sk is not needed for the decryption of c to get m . To solve this problem, the UC security in ROM also uses the hash list of H_1 , and search the list to find the right $(m, H_1(m))$ by testing the re-encryption relation $c = \text{LPKE.LEnc}(pk, pw, m; H_1(m))$. In QROM, this trick does not apply either. To solve the problem in QROM, we resort to the *on-line extraction* technique [20] so that the simulator can extract m while simulating H_1 in an indistinguishable way. Moreover, in case of active attack $(\tilde{c}, \tilde{C})$, after the simulator extracts a correct password pw , it also uses the on-line extraction technique to extract m' .

By instantiating LPKE and eLPKE in our generic construction, we obtain PAKE schemes $\text{PAKE}_{\text{lwe}}^{\text{RO}}$, $\text{PAKE}_{\text{ga}}^{\text{RO}}$, $\text{PAKE}_{\text{lwe}}^{\text{QRO}}$, $\text{PAKE}_{\text{ga}}^{\text{QRO}}$ from LWE in lattice, from GA-DDH in the isogeny setting, in ROM, and in QROM respectively.

The schematic overview of our PAKE constructions is given in Fig. 1.

Comparison. In Table 1, we compare our PAKE schemes with the available schemes based on post-quantum assumptions. Up to now, there are only two PAKE schemes [8, 31] with UC security from post-quantum assumptions, both of which are based on ICM. Note that ICM is equivalent to ROM [18]. Therefore, our work admits the *first* UC-secure PAKE schemes from isogenies, both in ROM and QROM, and the *first* UC-secure PAKE schemes from LWE in QROM. We also give more comparisons in terms of time complexity and communication complexity in the full version of our paper [25].

A recent progress on PAKE [26] shows how to compile PAKE to strong asymmetric PAKE with CSIDH assumption. Applying their compiler, our PAKE from GA-DDH can be upgraded to a strong asymmetric PAKE from isogenies.

2 Preliminaries

If x is defined by y or the value of y is assigned to x , we write $x := y$. For $\mu \in \mathbb{N}$, define $[\mu] := \{1, 2, \dots, \mu\}$. Denote by $x \leftarrow_{\$} \mathcal{X}$ the procedure of sampling x from set $\mathcal{X}$ uniformly at random. We also use “\$” to denote a random variable uniformly chosen from an implicitly known set. Let $|\mathcal{X}|$ denote the number of

elements in $\mathcal{X}$. All our algorithms are probabilistic unless stated otherwise. We use $y \leftarrow \mathcal{A}(x)$ to define the random variable y obtained by executing algorithm $\mathcal{A}$ on input x . We also use $y \leftarrow \mathcal{A}(x; r)$ to make explicit the random coins r used in the probabilistic computation. The notation $\approx_s$ represents statistical indistinguishability, while $\approx_c$ denotes computational indistinguishability. We use bold lower-case letters to denote column vectors. For a vector $\mathbf{v}$, we let $\|\mathbf{v}\|$ (resp., $\|\mathbf{v}\|_\infty$) denote its ℓ_2 (resp., ℓ_∞ infinity) norm.

2.1 Hardness Assumptions

Lattice Backgrounds. A q -ary lattice defined with $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ is $\Lambda(\mathbf{A}) := \{\mathbf{A}^T \mathbf{s} \mid \mathbf{s} \in \mathbb{Z}_q^n\} + q\mathbb{Z}^m$. The Gaussian function on $\mathbb{R}^n$ centered at $\mathbf{c}$ with parameter s is defined by $\rho_{s,\mathbf{c}}(\mathbf{x}) := e^{-\pi\|\mathbf{x}-\mathbf{c}\|^2/s^2}$. The discrete Gaussian distribution $D_{\Lambda,s,\mathbf{c}}$ over an n -dimensional lattice $\Lambda \subseteq \mathbb{R}^n$ is defined by $D_{\Lambda,s,\mathbf{c}}(\mathbf{x}) := \rho_{s,\mathbf{c}}(\mathbf{x})/\rho_{s,\mathbf{c}}(\Lambda)$ for any lattice vector $\mathbf{x} \in \Lambda$, where $\rho_{s,\mathbf{c}}(\Lambda) := \sum_{\mathbf{z} \in \Lambda} \rho_{s,\mathbf{c}}(\mathbf{z})$. The centered vector $\mathbf{c}$ is often omitted when $\mathbf{c} = \mathbf{0}$. More specifically, we use the notion $D_{\mathbb{Z}^m,r}$ to represent discrete Gaussian distribution over lattice $\Lambda := \mathbb{Z}^m$ centered at $\mathbf{c} := \mathbf{0}$ with parameter r .

Definition 1 (LWE Assumption [30]). Let $n, m, q \in \mathbb{N}$, and χ be a distribution over $\mathbb{Z}_q$. The $\text{LWE}_{n,q,m,\chi}$ assumption states that the following distributions are computationally indistinguishable: $(\mathbf{A}, \mathbf{A}^T \mathbf{s} + \mathbf{e}) \approx_c (\mathbf{A}, \mathbf{u})$, where $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}$, $\mathbf{e} \leftarrow \chi^m$, $\mathbf{s} \leftarrow \mathbb{Z}_q^n$ and $\mathbf{u} \leftarrow \mathbb{Z}_q^m$.

Lemma 1 ([4, 27]). There exists a PPT algorithm TrapGen that takes as input positive integers n, q ($q \geq 2$) and a sufficiently large $m = O(n \log q)$, outputs a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a trapdoor matrix $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m}$ such that $\mathbf{A}$ is statistically close to the uniform distribution, $\mathbf{A} \cdot \mathbf{T}_\mathbf{A} = \mathbf{0}$, and $\|\tilde{\mathbf{T}}_\mathbf{A}\| \leq O(\sqrt{n \log q})$, where $\tilde{\mathbf{T}}_\mathbf{A}$ denotes the Gram-Schmidt orthogonalization of $\mathbf{T}_\mathbf{A}$.

Lemma 2 ([27, Theorem 5.4]). There exists a deterministic polynomial-time algorithm Invert that takes as inputs the trapdoor information $\mathbf{T}_\mathbf{A}$ and a vector $\mathbf{v} := \mathbf{A}^T \cdot \mathbf{s} + \mathbf{e}$ with $\mathbf{s} \in \mathbb{Z}_q^n$ and $\|\mathbf{e}\| \leq q/(10 \cdot \sqrt{m})$, and outputs $\mathbf{s}$ and $\mathbf{e}$.

Lemma 3 ([21]). Let n and q be positive integers with q prime, and let $m \geq 2n \log q$. Then for all but a $2q^{-n}$ fraction of all $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and for any $r \geq \omega(\sqrt{\log m})$, the distribution of the syndrome $\mathbf{u} = \mathbf{A}\mathbf{e} \bmod q$ is statistically close to uniform over $\mathbb{Z}_q^n$, where $\mathbf{e} \leftarrow D_{\mathbb{Z}^m,r}$.

Cryptographic Group Actions. We will focus on group actions where $\mathbb{G}$ is abelian and the action is regular (we refer to the full version [25] for the definition). We recall the notion of restricted effective group actions (REGA) as follows.

Definition 2 (REGA [5]). A group action $(\mathbb{G}, \mathcal{X}, \star)$ is a restricted effective group action (REGA) if properties 1–4 are satisfied.

1. The group $\mathbb{G}$ is finite and generated by a set $\{g_1, \dots, g_n\}$ with $n = \text{poly}(\log |\mathbb{G}|)$.
2. The set $\mathcal{X}$ is finite and there exist PPT algorithms for membership testing and for computing unique representation of set element.
3. There exists a distinguished element $x_0 \in \mathcal{X}$, called the origin, such that its representation is known.
4. There exists an efficient algorithm that given g_i in the generating set and any $x \in \mathcal{X}$, outputs $g_i \star x$ and $g_i^{-1} \star x$ where $i \in [n]$.

With a REGA, we can use the generating set to approximate the random sampling process of $g \leftarrow_s \mathbb{G}$. The regularity of the $(\mathbb{G}, \mathcal{X}, \star)$ enables an efficient algorithm to sample $x \leftarrow_s \mathcal{X}$ uniformly.

There is a natural generalization of the DDH assumption in REGA settings.

Definition 3 (GA-DDH Assumptions). *Given a restricted effective group action $(\mathbb{G}, \mathcal{X}, \star)$, the Group Action DDH (GA-DDH) assumption states that the following distributions are computationally indistinguishable:*

$$\{x \leftarrow_s \mathcal{X}; s, t \leftarrow_s \mathbb{G} : (x, s \star x, t \star x, (s \cdot t) \star x)\} \approx_c \{x \leftarrow_s \mathcal{X}; s, t, z \leftarrow_s \mathbb{G} : (x, s \star x, t \star x, z \star x)\}.$$

We can instantiate REGA with isogeny-based group actions, like CSIDH. The GA-DDH assumption is believed to hold for CSIDH [17].

2.2 UC Framework for PAKE

We present a concise overview of the UC framework for PAKE. Figure 4 shows the picture of the “real world” execution of a protocol Π in environment $\mathcal{Z}$ and the “ideal world” execution with a simulator Sim in environment $\mathcal{Z}$. The environment $\mathcal{Z}$ can be considered as higher-level protocols utilizing Π as a sub-protocol ($\mathcal{Z}$ also includes the adversary that is attacking those higher-level protocols). The adversary $\mathcal{A}$ essentially models a completely insecure network and it communicates continuously with $\mathcal{Z}$. The client/server instances send and receive messages via $\mathcal{A}$, and $\mathcal{A}$ can do dropping, injecting, and modifying protocol messages at will.

In the real world, the client/server instances are executed as described in the protocol Π . They receive their inputs (passwords in PAKE) from $\mathcal{Z}$ and send their outputs (session key in PAKE) to $\mathcal{Z}$. Hash function H is modeled as a random oracle. Clients, servers and adversary may directly (quantum) access the random oracle H . However, $\mathcal{Z}$ can only access H indirectly via $\mathcal{A}$.

In the ideal world, clients/servers are “dummy” parties that pass their passwords directly from $\mathcal{Z}$ to an ideal functionality $\mathcal{F}_{\text{pake}}$, and their outputs directly from $\mathcal{F}_{\text{pake}}$ to $\mathcal{Z}$. We mainly follow the definition of $\mathcal{F}_{\text{pake}}$ by Shoup [32], which is a modified version by Canetti et al. [16]. Simulator Sim can communicate with $\mathcal{Z}$, just as $\mathcal{A}$ did in the real world. Besides, Sim can also interact with $\mathcal{F}_{\text{pake}}$. Sim has two main tasks. Firstly, it must simulate the network transcripts in a way that is indistinguishable from those generated in the real world. Secondly, Sim must provide appropriate inputs to $\mathcal{F}_{\text{pake}}$ to obtain outputs (session key) that are indistinguishable from the outputs produced by the client/server instances in the real world.

Functionality $\mathcal{F}_{pake}$	
The functionality $\mathcal{F}_{pake}$ is parameterized by a security parameter λ . It interacts with a simulator Sim and a set of parties via the following queries:	
Password Storage	
Upon receiving a query (StorePWFile, $C^{(i)}, S^{(j)}, pw$) from a client $C^{(i)}$ or a server $S^{(j)}$:	
If there exists a record $\langle \text{file}, C^{(i)}, S^{(j)}, \cdot \rangle$, ignore this query. Otherwise, record $\langle \text{file}, C^{(i)}, S^{(j)}, pw \rangle$, and send (StorePWFile, $C^{(i)}, S^{(j)}$) to Sim.	
Sessions	
Upon receiving a query (NewClient, $iid, S^{(j)}$) from a client $C^{(i)}$:	
Retrieve the record $\langle \text{file}, C^{(i)}, S^{(j)}, pw \rangle$. Send (NewClient, $C^{(i)}, iid, S^{(j)}$) to Sim.	
Record $(C^{(i)}, iid, S^{(j)}, pw)$ and mark it as fresh .	
In this case, $S^{(j)}$ is called the intended partner of $(C^{(i)}, iid)$.	
Upon receiving a query (NewServer, $iid', C^{(i)}$) from a server $S^{(j)}$:	
Retrieve the record $\langle \text{file}, C^{(i)}, S^{(j)}, pw \rangle$. Send (NewServer, $S^{(j)}, iid', C^{(i)}$) to Sim.	
Record $(S^{(j)}, iid', C^{(i)}, pw)$ and mark it as fresh .	
In this case, $C^{(i)}$ is called the intended partner of $(S^{(j)}, iid')$.	
Two instances $(C^{(i)}, iid)$ and $(S^{(j)}, iid')$ are said to be partnered , if there are two records $(C^{(i)}, iid, S^{(j)}, pw)$ and $(S^{(j)}, iid', C^{(i)}, pw)$ sharing the same pw .	
Active Session Attacks	
Upon receiving a query (Testpw, P, iid, pw') from Sim:	
If there is a fresh record $(P, iid, \cdot, pw)$:	
If $pw' = pw$, mark the record compromised and reply to Sim with “correct guess”.	
If $pw' \neq pw$, mark the record interrupted and reply with “wrong guess”.	
Key Generation	
Upon receiving a query (FreshKey, P, iid, sid) from Sim:	
If 1) there is a fresh record (P, iid, Q, pw) ; and 2) sid has never been assigned to P 's any other instance (P, iid') :	
Pick a new random key $sKey$, mark the record (P, iid, Q, pw) as completed , assign it with sid , send $(iid, sid, sKey)$ to P , and record $(P, Q, sid, sKey)$.	
Upon receiving a query (CopyKey, P, iid, sid) from Sim:	
If 1) there is a fresh record (P, iid, Q, pw) and a completed record (Q, iid^*, P, pw) s.t. (P, iid) and (Q, iid^*) are partnered; and 2) sid has never been assigned to P 's any other instance (P, iid') ; and 3) the partnered (Q, iid^*) is the unique one that has been assigned with sid :	
Retrieve the record $(Q, P, sid, sKey)$, mark the record (P, iid, Q, pw) as completed , assign it with sid , and send $(iid, sid, sKey)$ to P .	
Upon receiving a query (CorruptKey, $P, iid, sid, sKey$) from Sim:	
If 1) there is a compromised record (P, iid, Q, pw) ; and 2) sid has never been assigned to P 's any other instance (P, iid') :	
Mark the record (P, iid, Q, pw) as completed , assign it with sid , and send $(iid, sid, sKey)$ to P .	
Upon receiving a query (Abort, P, iid) from Sim:	
mark the record $(P, iid, \cdot, pw)$ as abort , and send $(iid, \perp)$ to P .	

Fig. 3. The ideal functionality $\mathcal{F}_{pake}$ for PAKE.

The ideal functionality $\mathcal{F}_{pake}$ is shown in Fig. 3. We stress that our ideal functionality $\mathcal{F}_{pake}$ supports *mutual authentication*, which indicates mutual explicit authentication. It is mainly captured by the first item and the last item of “Key Generation” in our ideal functionality (Fig. 3), which guarantee that if an adversary makes an unsuccessful password guess on a protocol instance, then when that instance terminates, the corresponding party will receive an abort message. Accordingly, this requires that each party can identify active attacks explicitly in the real-world protocol.

In a nutshell, we say protocol Π securely emulates ideal functionality $\mathcal{F}_{pake}$ if for any efficient adversary $\mathcal{A}$, there exists an efficient simulator Sim such that no efficient environment $\mathcal{Z}$ can effectively distinguish between the actual execution in the real world and the hypothetical execution in the ideal world.

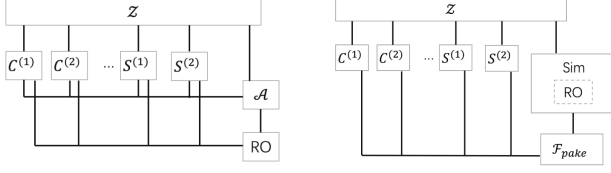


Fig. 4. The real world execution (left) and the ideal world execution (right).

2.3 ROM vs. QROM

In the Random Oracle Model (ROM), a cryptographic hash function $H : \mathcal{X} \rightarrow \mathcal{Y}$ is idealized as a truly random function $\text{RF} : \mathcal{X} \rightarrow \mathcal{Y}$. And any adversary needs to query H on inputs $x \in \mathcal{X}$ to learn the hash values $H(x)$.

In the quantum world, a quantum algorithm $\mathcal{A}$ can perform superposition queries to the random oracle H , and then oracle H behaves as a unitary operation $|x\rangle|y\rangle \mapsto |x\rangle|y \oplus H(x)\rangle$. In this case, H becomes a quantum random oracle (QRO). The QRO model supports classical queries x on H , and this can be formalized as setting query register and output register to be $|x\rangle|0\rangle$, and measuring the output register after the unitary operation $|x\rangle|0\rangle \mapsto |x\rangle|0 \oplus H(x)\rangle$.

3 PAKE from Basic LPKE in ROM

In this section we present the definition of basic LPKE and show how to apply the FO-transformation to LPKE to obtain a CCA-secure KEM. Then we show the generic construction of PAKE from basic LPKE and its UC security in ROM.

3.1 Basic Lossy Public Key Encryption (LPKE)

LPKE works in two modes. If label b is a normal one, then LPKE works in normal mode and has correctness and CPA security. If label b is a lossy one, then LPKE works in lossy mode, and the ciphertexts are random. With the trapdoor, algorithm IsLossy can decide whether label b is lossy or normal for pk .

Definition 4 (Basic LPKE). A basic Lossy Public Key Encryption scheme $\text{LPKE} = (\text{LSetup}, \text{LKeyGen}, \text{LEnc}, \text{LDec}, \text{IsLossy})$ consists of five probabilistic algorithms.

- $\text{LSetup}(1^\lambda)$: The setup algorithm takes as input the security parameter 1^λ , and outputs a public parameter pp and a trapdoor td . The parameter pp specifies a public key space $\mathcal{PK}$, a secret key space $\mathcal{SK}$, a label space $\mathcal{T}$, a message space $\mathcal{M}$ and a ciphertext space $\mathcal{CT}$. All the remaining algorithms take pp as input, and we omit it for simplicity.
- $\text{LKeyGen}(b)$: The key generation algorithm takes as input a label b and outputs a key pair (pk, sk) .

- $\text{LEnc}(pk, b, m)$: The encryption algorithm takes as input a public key pk , a label b and a message m , and outputs a ciphertext c .
- $\text{LDec}(sk, c)$: The decryption algorithm takes as input a secret key sk and a ciphertext c , and outputs a message m .
- $\text{IsLossy}(td, pk, b)$: The algorithm takes as input a trapdoor td , a public key pk and a label b , and outputs a bit 0 or 1.

For any $pk \in \mathcal{PK}$ and $b \in \mathcal{T}$, if $\text{IsLossy}(td, pk, b) = 1$, then label b is called a *lossy label* of public key pk . Otherwise, label b is called a *normal label* of pk .

Correctness of Basic LPKE. For all $b \in \mathcal{T}$ and $m \in \mathcal{M}$, it holds that

$$\Pr[\text{LDec}(sk, \text{LEnc}(pk, b, m)) \neq m] \leq \text{negl}(\lambda), \Pr[\text{IsLossy}(td, pk, b) = 1] \leq \text{negl}(\lambda),$$

where $(pp, td) \leftarrow \text{LSetup}(1^\lambda)$ and $(pk, sk) \leftarrow \text{LKeyGen}(b)$.

A basic LPKE scheme LPKE should satisfy the following properties.

① **Pseudorandomness of Public Key.** For all $b \in \mathcal{T}$, it holds that

$$\left\{ \begin{array}{l} (pp, td) \leftarrow \text{LSetup} \\ (pk, sk) \leftarrow \text{LKeyGen}(b) \end{array} : (pp, pk) \right\} \approx_c \left\{ \begin{array}{l} (pp, td) \leftarrow \text{LSetup} \\ pk \leftarrow_s \mathcal{PK} \end{array} : (pp, pk) \right\}.$$

- ② **Random Ciphertexts under Lossy Labels.** For all admissible adversary $\mathcal{A}$ which outputs (pk, b) such that $\text{IsLossy}(td, pk, b) = 1$, it holds that $(pp, pk, b, m, c) \approx_s (pp, pk, b, m, c')$, where $(pp, td) \leftarrow \text{LSetup}$, $(pk, b) \leftarrow \mathcal{A}(pp)$ s.t. $\text{IsLossy}(td, pk, b) = 1$, $m \leftarrow_s \mathcal{M}$, $c \leftarrow \text{LEnc}(pk, b, m)$ and $c' \leftarrow_s \mathcal{CT}$.
- ③ **Uniqueness of Normal Labels among Polynomial-Size Set:** For any $Q := \text{poly}(\lambda)$, it holds that

$$\Pr \left[\begin{array}{l} (pp, td) \leftarrow \text{LSetup} \\ b_1, \dots, b_Q \leftarrow_s \mathcal{T} \end{array} : \begin{array}{l} \exists pk \in \mathcal{PK}, i \neq j \text{ with } i, j \in [Q] \\ \text{IsLossy}(td, pk, b_i) = 0 \wedge \text{IsLossy}(td, pk, b_j) = 0 \end{array} \right] \leq \text{negl}(\lambda).$$

④ **Lossiness of Random Labels.** For all adversary $\mathcal{A}$, it holds that

$$\Pr[(pp, td) \leftarrow \text{LSetup}; pk \leftarrow \mathcal{A}(pp); b \leftarrow_s \mathcal{T} : \text{IsLossy}(td, pk, b) = 0] \leq \text{negl}(\lambda).$$

- ⑤ **Ciphertext Unpredictability under Normal Labels:** For all but a negligible fraction of pp from $(pp, td) \leftarrow \text{LSetup}(1^\lambda)$, for all $b \in \mathcal{T}$, all $(pk, sk) \leftarrow \text{LKeyGen}(b)$, all messages $m \in \mathcal{M}$ and all ciphertexts $c \in \mathcal{CT}$, it holds that $\Pr[r \leftarrow_s \mathcal{R} : \text{LEnc}(pk, b, m; r) = c] \leq \text{negl}(\lambda)$.
- ⑥ **CPA Security under Normal Labels:** For all PPT $\mathcal{A}$, we have $(pp, pk, b, c_0) \approx_c (pp, pk, b, c_1)$, where $(pp, td) \leftarrow \text{LSetup}$, $(b, st) \leftarrow \mathcal{A}(pp)$, $(pk, sk) \leftarrow \text{LKeyGen}(b)$, $(m_0, m_1) \leftarrow \mathcal{A}(st, pk)$, $c_0 \leftarrow \text{LEnc}(pk, b, m_0)$, $c_1 \leftarrow \text{LEnc}(pk, b, m_1)$.

Remark 1. The concept of R -lossy PKE (R -LPKE) was introduced by Boyle et al. [14]. Our basic LPKE can be considered as a special R -LPKE with relation R simply defined as $R(K, t) := (K \stackrel{?}{=} t)$, but our basic LPKE is augmented with IsLossy algorithm and equipped with a different set of properties.

By leveraging the FO-transformation, LPKE under normal label b can be transformed to a labeled KEM scheme, whose encapsulation algorithm is implemented as $c \leftarrow \text{LEnc}(pk, b, m; H_1(m))$ with $m \leftarrow_s \mathcal{M}$ and $K := H_2(m)$ (See the full version [25] for its formal description). Consequently, the CPA-security of LPKE is upgraded to CCA-security of the KEM, according to [20]. This is shown in Lemma 4. Meanwhile, the property of *random ciphertexts under lossy labels* for LPKE is inherited by that of KEM but degrades to a pseudo-random one, as shown in Lemma 5 with proof in the full version [25].

Lemma 4 (CCA Security of KEM [20]). *Suppose that $\text{LPKE} = (\text{LSetup}, \text{LKeyGen}, \text{IsLossy}, \text{LEnc}, \text{LDec})$ is a basic lossy public key encryption scheme and H_1 and H_2 are two (quantum-accessible) random oracles. For any PPT adversary $\mathcal{A}$ against CCA security of the KEM scheme, it holds that*

$$\text{Adv}_{\text{KEM}}^{\text{CCA-FO}}(\mathcal{A}) := \left| \Pr \left[\text{Exp}_{\text{KEM}}^{\text{CCA-FO-0}} \Rightarrow 1 \right] - \Pr \left[\text{Exp}_{\text{KEM}}^{\text{CCA-FO-1}} \Rightarrow 1 \right] \right| \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\text{KEM}}^{\text{CCA-FO-}\beta}$ is defined in Fig 5.

$\text{Exp}_{\text{KEM}}^{\text{CCA-FO-}\beta} :$	$\beta \in \{0, 1\}$
$(\text{pp}, td) \leftarrow \text{LSetup}; (b, \text{st}) \leftarrow \mathcal{A}(\text{pp})$ $(pk, sk) \leftarrow \text{LKeyGen}(b)$ $m \leftarrow_s \mathcal{M}, c^* \leftarrow \text{LEnc}(pk, b, m; H_1(m))$ $K_0^* := H_2(m), K_1^* \leftarrow_s \mathcal{K}_{\text{KEM}}$ $\beta' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Dec}}(\cdot)}(\text{st}, pk, c^*, K_{\beta}^*)$ Return β'	$\mathcal{O}_{\text{Dec}}(c):$ If $c = c^*$: Return $\perp$ $K \leftarrow \text{LDec}(sk, c)$ Return K

Fig. 5. CCA-security of KEM from LPKE via FO

Lemma 5 (Ciphertext Pseudo-Randomness under Lossy Labels). *Suppose that $\text{LPKE} = (\text{LSetup}, \text{LKeyGen}, \text{IsLossy}, \text{LEnc}, \text{LDec})$ is a basic lossy public key encryption scheme and H_1 and H_2 are two (quantum-accessible) random oracles. For all admissible adversary $\mathcal{A}$ which outputs (pk, b) such that $\text{IsLossy}(td, pk, b) = 1$, it holds that $(\text{pp}, pk, b, c, K) \approx_c (\text{pp}, pk, b, c' \leftarrow_s \mathcal{CT}, K' \leftarrow_s \mathcal{K})$, where $(\text{pp}, td) \leftarrow \text{LSetup}, (pk, b) \leftarrow \mathcal{A}(\text{pp})$ s.t. $\text{IsLossy}(td, pk, b) = 1$, $m \leftarrow_s \mathcal{M}$, $c \leftarrow \text{LEnc}(pk, b, m; H_1(m))$, and $K := H_2(m)$.*

3.2 Construction of PAKE from Basic LPKE in ROM

We propose a generic construction of 3-round PAKE scheme PAKE^{RO} from LPKE and PKE. The underlying building blocks are as follows.

- a basic lossy public key encryption scheme $\text{LPKE} = (\text{LSetup}, \text{LKeyGen}, \text{LEnc}, \text{LDec}, \text{IsLossy})$ with message space $\{0, 1\}^\lambda$, label space $\mathcal{T}$ and randomness space $\mathcal{R}$;
- a CCA-secure public key encryption scheme $\text{PKE} = (\text{CKeyGen}, \text{CEnc}, \text{CDec})$;
- four hash functions: $H : \mathcal{PW} \rightarrow \mathcal{T}$, $H_1 : \{0, 1\}^\lambda \rightarrow \mathcal{R}$, $H_2 : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{3\lambda}$, $H_3 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$.

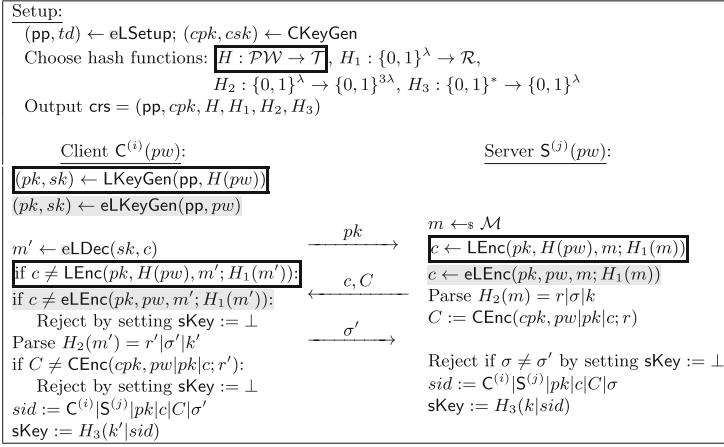


Fig. 6. Construction of PAKE^{RO} (resp. PAKE^{QRO}) from LPKE (resp. eLPKE). text only appears in PAKE^{RO} from LPKE and text only appears in PAKE^{QRO} from eLPKE .

Our generic construction of PAKE from LPKE and PKE is given in Fig. 6.

The UC security for PAKE^{RO} constructed from LPKE in ROM is shown in Theorem 1. To facilitate the proof, we define the concept of *linked to* like [32].

Definition 5 (Linked To). *In a protocol execution, a client (resp. server) instance is “linked to” a server (resp. client) instance at a specific time being if the transcript of the client (resp. server) is consistent to that of the server (resp. client), i.e., the messages received and sent by a party are exactly those messages sent and received by another party.*

Theorem 1. *If LPKE is a basic LPKE scheme, PKE is a CCA-secure PKE scheme, and H, H_1, H_2, H_3 work as random oracles, then the PAKE scheme PAKE^{RO} in Fig. 6 securely emulates $\mathcal{F}_{\text{pake}}$, hence achieving UC security in ROM.*

Proof. The main objective of the proof is to construct a PPT simulator Sim . Sim is designed to have access to the ideal functionality $\mathcal{F}_{\text{pake}}$ and interact with the environment $\mathcal{Z}$, thereby emulating the real-world PAKE protocol interactions involving the adversary $\mathcal{A}$, the parties, and the environment $\mathcal{Z}$. It is important to note that Sim does not possess any password.

The complete description of the simulator Sim is given in the full version [25].

Let $\mathbf{Real}_{\mathcal{Z}, \mathcal{A}}$ represent the real-world experiment where the environment $\mathcal{Z}$ interacts with the actual parties and adversary $\mathcal{A}$, while $\mathbf{Ideal}_{\mathcal{Z}, \text{Sim}}$ represents the ideal experiment where $\mathcal{Z}$ interacts with the simulator Sim .

Our goal is to demonstrate that $|\Pr[\mathbf{Real}_{\mathcal{Z}, \mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{Ideal}_{\mathcal{Z}, \text{Sim}} \Rightarrow 1]|$ is negligible by employing a series of games, denoted as Game G_0 - G_{11} . In this sequence, G_0 corresponds to $\mathbf{Real}_{\mathcal{Z}, \mathcal{A}}$, while G_{11} corresponds to $\mathbf{Ideal}_{\mathcal{Z}, \text{Sim}}$. We aim to show that these adjacent games are indistinguishable from the view of $\mathcal{Z}$.

Game G_0 . This is the real experiment $\mathbf{Real}_{\mathcal{Z}, \mathcal{A}}$. In this experiment, $\mathcal{Z}$ initializes a password for each client-server pair, sees the interactions among clients,

servers and adversary $\mathcal{A}$, and also obtains the corresponding session keys of protocol instances. Here $\mathcal{A}$ may implement attacks like view, modify, insert, or drop messages over the network. We have $\Pr[\mathbf{Real}_{\mathcal{Z}, \mathcal{A}} \Rightarrow 1] = \Pr[\mathbf{G}_0 \Rightarrow 1]$.

Game $\mathbf{G}_1$ (simulations for clients and servers with pw).

In this game, we introduce a simulator $\mathbf{Sim}$ who *receives passwords* from $\mathcal{Z}$. Then it simulates the clients and servers to generate transcripts for instances of the PAKE protocol. With the knowledge of *passwords*, the simulations of the behaviors of all clients and servers are perfect.

Moreover, $\mathbf{Sim}$ also simulates random oracles H, H_1, H_2, H_3 by maintaining four separate lists, namely $\mathcal{L}_H, \mathcal{L}_{H_1}, \mathcal{L}_{H_2}, \mathcal{L}_{H_3}$. For example, for a query x on $H(\cdot)$, if $(x, y) \in \mathcal{L}_H$, then $\mathbf{Sim}$ will return y as the reply. Otherwise, $\mathbf{Sim}$ will choose a random element y , record (x, y) in $\mathcal{L}_H$, and return y as the reply. By the ideal functionality of random oracles, $\mathbf{Sim}$'s simulations for oracles H, H_1, H_2, H_3 are also perfect. So we have $\Pr[\mathbf{G}_1 \Rightarrow 1] = \Pr[\mathbf{G}_0 \Rightarrow 1]$.

The following games will change the simulations of $\mathbf{Sim}$ step by step in an indistinguishable way so that $\mathbf{Sim}$ can arrive at its final form and accomplish the simulations in $\mathbf{Ideal}_{\mathcal{Z}, \mathbf{Sim}}$ without passwords pw .

Game $\mathbf{G}_2$ (simulation for crs). In $\mathbf{G}_2$, $\mathbf{Sim}$ simulates the generation of $\text{crs} = (\text{pp}, \text{cpk}, \dots)$ with $(\text{pp}, \text{td}) \leftarrow \text{LSetup}$ and $(\text{cpk}, \text{csk}) \leftarrow \text{CKeyGen}$, and it also records the trapdoor td of LPKE and the secret key csk of PKE. Clearly, the simulation of crs is perfect, so we have $\Pr[\mathbf{G}_2 \Rightarrow 1] = \Pr[\mathbf{G}_1 \Rightarrow 1]$.

Game $\mathbf{G}_3$ (simulation of $r|\sigma|k$ for server instances and simulation of the third-round message σ' for client instances in case of passive attacks). In $\mathbf{G}_3$, simulator $\mathbf{Sim}$ is the same as in $\mathbf{G}_2$, except for the simulation of generating $r|\sigma|k$ for server instances and the corresponding simulation for client instances in case of passive attacks.

- For a server instance $(\mathbf{S}^{(j)}, iid')$ that is linked to $(\mathbf{C}^{(i)}, iid)$ when receiving a first-round message pk , simulator $\mathbf{Sim}$ will randomly sample $r|\sigma|k \leftarrow \{0, 1\}^{3\lambda}$, rather than computing $r|\sigma|k := H_2(m)$ as did in $\mathbf{G}_2$. Note that pk must have been generated for the client instance $(\mathbf{C}^{(i)}, iid)$ by $\mathbf{Sim}$.
- For a client instance $(\mathbf{C}^{(i)}, iid)$ that is linked to server instance $(\mathbf{S}^{(j)}, iid')$ when receiving a second-round message (c, C') , we know that (c, C') and the corresponding $r|\sigma|k$ must have been generated for $(\mathbf{S}^{(j)}, iid')$ by $\mathbf{Sim}$. In this case, $\mathbf{Sim}$ directly sets $\text{sKey} := H_3(k|sid)$ and outputs $\sigma' := \sigma$.

According to Lemma 4, $(pk, c = \text{LEnc}(pk, H(pw), m; H_1(m)), r|\sigma|k := H_2(m))$ works as a CCA-secure KEM, where pk is the public key, $H(pw)$ is the normal label of pk , c is the ciphertext, $m \leftarrow_{\$} \mathcal{M}$ and $H_2(m) = r|\sigma|k$ is the encapsulated key. Then by the CCA-security of KEM we have

$$(pk, c \leftarrow \text{LEnc}(pk, m; H_1(m)), r|\sigma|k := H_2(m)) \approx_c (pk, c \leftarrow \text{LEnc}(pk, m; H_1(m)), \$)$$

for PPT adversaries access to decryption oracle. There are at most ℓ sessions, so hybrid arguments across the ℓ sessions yield

$$|\Pr[G_3 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]| \leq \ell \cdot \text{Adv}_{\text{KEM}}^{\text{CCA-FO}}(\mathcal{B}_{\text{KEM}}) \leq \text{negl}(\lambda).$$

Game G_4 (simulation of sKey in case of passive attacks). In G_4 , Sim is the same as in G_3 , except for the generation of sKey for client instances and its corresponding sever instances in case of passive attacks.

- For a client instance $(C^{(i)}, iid)$ that is linked to server instance $(S^{(j)}, iid')$, when client $C^{(i)}$ receives a second-round message (c, C') , simulator Sim changes the simulation of generating session key sKey for $C^{(i)}$. More precisely, Sim will not set $\text{sKey} := H_3(k|sid)$ as did in G_3 , but sample $\text{sKey} \leftarrow_{\$} \{0, 1\}^\lambda$ instead. Sim stores sKey for $(C^{(i)}, iid)$. Note that (c, C') must have been generated by Sim and $r|\sigma|k$ sampled uniformly by Sim for server instance $(S^{(j)}, iid')$. For the simulation of $C^{(i)}$ outputting σ' , Sim still outputs $\sigma' := \sigma$ by retrieving σ from $r|\sigma|k$, just like G_3 .
- For a server instance $(S^{(j)}, iid')$ that is linked to $(C^{(i)}, iid)$, when $S^{(j)}$ receives a third-round message σ' , Sim will compare σ' with σ in the transcription (pk, c, C, σ) of instance $(S^{(j)}, iid')$. If $\sigma' = \sigma$, Sim will retrieve sKey stored for $(C^{(i)}, iid)$ and set the same session key sKey for $(S^{(j)}, iid')$.

Note that in the above two cases, $(C^{(i)}, iid)$ and $(S^{(j)}, iid')$ share the same $r|\sigma|k$, where $r|\sigma|k$ is uniformly chosen and independent of the view of $\mathcal{A}$. The uniformity of k makes sure that $\mathcal{A}$ ever queries $H_3(k|sid)$ for some sid with negligible probability. As long as no query on $H_3(k|\cdot)$ is made by $\mathcal{A}$, the session key $\text{sKey} := H_3(k|\cdot)$ is uniform and independent of other variables in G_3 . In G_4 , the session key sKey is sampled in a random and independent way. Consequently, G_4 and G_3 are the same to $\mathcal{Z}$, except that $\mathcal{A}$ ever queries $H_3(k|\cdot)$ which happens with negligible probability. So we have $|\Pr[G_4 \Rightarrow 1] - \Pr[G_3 \Rightarrow 1]| \leq \text{negl}(\lambda)$.

Game G_5 (simulation for client instances in case of active attacks). In G_5 , simulator Sim is the same as in G_4 , except that Sim will add a rejection rule in the simulations for client instances in case of active attacks. More precisely,

- For a client instance $(C^{(i)}, iid)$ that is not linked to any server instance $(S^{(j)}, iid')$ when receiving the second-round message $(\tilde{c}, \tilde{C})$, we know that $(\tilde{c}, \tilde{C})$ is NOT generated from any server instance, so it must be forged by adversary $\mathcal{A}$ (with active attacks). Let pw be the password shared between $C^{(i)}$ and $S^{(j)}$ and pk be the public key generated by Sim for instance $(C^{(i)}, iid)$.

Rejection rule: Upon receiving the second-round message $(\tilde{c}, \tilde{C})$, Sim first extracts password pw' by invoking $pw'|pk'|c' \leftarrow \text{CDec}(csk, \tilde{C})$. If $pw \neq pw'$ or $pk'|c' \neq pk|\tilde{c}$,

then Sim rejects $(\tilde{c}, \tilde{C})$ by setting $\text{sKey} := \perp$.

G_5 and G_4 differ only when a message $(\tilde{c}, \tilde{C})$ satisfying $(\star)$ leads to rejection in G_5 but not in G_4 . However, if $(\tilde{c}, \tilde{C})$ satisfies $(\star)$ then either $pw' \neq pw$ or $pk'|c' \neq pk|\tilde{c}$. If such $(\tilde{c}, \tilde{C})$ does not lead to rejection in G_4 , then we have $\tilde{C} = \text{CEnc}(cpk, pw|pk|\tilde{c}; r)$ and $pw'|pk'|c' = \text{CDec}(csk, \tilde{C})$, but $pw'|pk'|c' \neq pw|pk|\tilde{c}$,

which contradicts to the correctness of PKE. Therefore, $(\tilde{c}, \tilde{C})$ satisfying $(\star)$ must lead to rejection in G_4 except with negligible probability, and hence we have

$$|\Pr[G_5 \Rightarrow 1] - \Pr[G_4 \Rightarrow 1]| \leq \text{negl}(\lambda).$$

Game G_6 (Get Rid of sk in simulation for client instances in case of active attacks). In G_6 , Sim is the same as in G_5 , except for the generation of m' during Sim's simulations for client instances in case of active attacks.

- For any client instance $(C^{(i)}, iid)$ that is not linked to any server instance $(S^{(j)}, iid')$ when receiving the second-round message $(\tilde{c}, \tilde{C})$, we know that $(\tilde{c}, \tilde{C})$ is NOT generated from any server instance, so it must be forged by adversary $\mathcal{A}$. When generating m' during the simulation for instance $(C^{(i)}, iid)$, Sim will not use the decryption algorithm to obtain $m' \leftarrow \text{LDec}(sk, \tilde{c})$ as did in G_5 . Instead, it will check whether $\exists(m', r_H) \in \mathcal{L}_{H_1}$ such that $\tilde{c} = \text{LEnc}(pk, H(pw), m'; r_H)$, where pw is the password of $C^{(i)}$ and pk is the first-round message generated by Sim for $(C^{(i)}, iid)$. If there exists such pair $(m', r_H) \in \mathcal{L}_{H_1}$, then Sim retrieves m' as the decrypted plaintext. Otherwise Sim rejects $(\tilde{c}, \tilde{C})$ by setting $sKey := \perp$.

We consider two cases.

Case I. $\exists(m', r_H) \in \mathcal{L}_{H_1}$ s.t. $\tilde{c} = \text{LEnc}(pk, H(pw), m'; r_H)$. By the correctness of LPKE, we have $\text{LDec}(sk, \tilde{c}) = m'$. Therefore, G_6 results in the same m' as that in G_5 , and thus G_6 and G_5 are indistinguishable to $\mathcal{Z}$ in this case.

Case II. $\nexists(m', r_H) \in \mathcal{L}_{H_1}$ s.t. $\tilde{c} = \text{LEnc}(pk, H(pw), m'; r_H)$. This suggests that adversary $\mathcal{A}$ does not ever query $H_1(m')$ and hence $H_1(m')$ is random. Recall that pk was generated from $\text{LKeyGen}(H(pw))$, so $H(pw)$ is the normal label of pk . Then by the property of *ciphertext unpredictability under normal labels*, $\tilde{c} = \text{LEnc}(pk, H(pw), m'; H_1(m'))$ hardly holds. Accordingly, Sim's simulation of $C^{(i)}$ will reject with $sKey := \perp$ in G_5 , except with negligible probability.

In G_6 , Sim will terminate the simulation by setting $sKey := \perp$. Obviously, G_6 and G_5 are identical to $\mathcal{Z}$ except with negligible probability in this case.

Therefore, we have $|\Pr[G_6 \Rightarrow 1] - \Pr[G_5 \Rightarrow 1]| \leq \text{negl}(\lambda)$.

We stress that now in G_6 (and hereafter), Sim's simulation for client instances does not need the secret key sk of LPKE any more, no matter dealing with active attacks or passive attacks. This helps us to proceed to the next game.

Game G_7 (simulation of generating first-round message pk without pw). In G_7 , Sim is the same as in G_6 , except for Sim's simulation of generating the first-round message pk for client instances.

- For any client instance $(C^{(i)}, iid)$, when generating the first-round message pk , Sim randomly samples $pk \leftarrow_s \mathcal{PK}$ in G_7 , rather than invoking $(pk, sk) \leftarrow \text{LKeyGen}(H(pw))$ as did in G_6 .

Due to the *pseudo-randomness of public key* of LPKE and by hybrid arguments across the ℓ sessions, we have $|\Pr[G_7 \Rightarrow 1] - \Pr[G_6 \Rightarrow 1]| \leq \text{negl}(\lambda)$.

We note that the reduction proof for the above equation proceeds smoothly since sk is not needed any more in the simulation for client instances.

Game G_8 (simulation of generating c in second-round message). In G_8 , Sim is the same as in G_7 , except for Sim 's simulation of generating c in the second-round message (c, C) for server instances. There are two cases.

Case 1: Passive attacks on Servers. For a server instance $(S^{(j)}, iid')$ that is linked to some client instance $(C^{(i)}, iid)$ when receiving a first-round message pk , simulator Sim will sample c by $c \leftarrow_s \mathcal{CT}$, rather than computing it with $c \leftarrow \text{LEnc}(pk, H(pw), m; H_1(m))$ as did in G_7 .

Note that $r|\sigma|k \leftarrow_s \{0, 1\}^{3\lambda}$ and $C \leftarrow \text{CEnc}(cpk, pw|pk|c; r)$ are still computed in the same way as in G_7 .

Case 2: Active attacks on Servers. For a server instance $(S^{(j)}, iid')$ that is not linked to any client instance when receiving a first-round message pk , we further consider three sub-cases.

Case 2.1: $\nexists (pw', r_H) \in \mathcal{L}_H$ s.t. $\text{IsLossy}(td, \tilde{pk}, r_H) = 0$. In this case, Sim will compute $c \leftarrow_s \mathcal{CT}$, $r|\sigma|k \leftarrow_s \{0, 1\}^{3\lambda}$, rather than computing $c \leftarrow \text{LEnc}(pk, H(pw), m; H_1(m))$ and $r|\sigma|k := H_2(m)$ as did in G_7 .

Case 2.2: $\exists! (pw', r_H) \in \mathcal{L}_H$ s.t. $\text{IsLossy}(td, \tilde{pk}, r_H) = 0$. In this case, Sim extracts this password pw' and checks whether $pw' = pw$ or not.

- If $pw' \neq pw$, Sim simulates (c, C) just like Case 2.1.

- If $pw' = pw$, Sim computes (c, C) just like G_7 , i.e., $c \leftarrow \text{LEnc}(\tilde{pk}, H(pw), m; H_1(m))$ and $C \leftarrow \text{CEnc}(cpk, pw|pk|c; r)$.

Case 2.3: $\exists (pw, r_H), (pw', r'_H) \in \mathcal{L}_H$ s.t. $\text{IsLossy}(td, pk, r_H) = \text{IsLossy}(td, \tilde{pk}, r'_H) = 0$. In this case, Sim just aborts the simulation directly.

In Case 1, pk is random and independent of password pw . Then $H(pw)$ is a random label w.r.t. pk . According to the *lossiness of random labels*, we have $\text{IsLossy}(td, pk, H(pw)) = 1$, i.e., $H(pw)$ is a lossy label of pk . Then according to Lemma 5, the ciphertext $c := \text{LEnc}(pk, H(pw), m; H_1(m))$ is pseudo-random in G_7 . Therefore, we can replace c with a random one as did in G_8 in a computationally indistinguishable way.

In Case 2.1, $\mathcal{A}$ did not ever query $H(pw)$, and hence $H(pw)$ is random to $\mathcal{A}$. With a similar argument as Case 1, we can also replace c with a random one. Now that c is independent of m , then the randomness of m makes sure that $\mathcal{A}$ hardly ever queries $H_2(m)$. So, $r|\sigma|k := H_2(m)$ is uniform and independent of c .

In Case 2.2, $H(pw')$ is a normal label of pk . If pw' is not the correct password, then $H(pw)$ must be a lossy label to $\tilde{pk}$ except with a negligible probability. The reason is as follows. If $(pw, \cdot) \in \mathcal{L}_H$, then $\text{IsLossy}(td, \tilde{pk}, H(pw)) = 1$ must hold, hence $H(pw)$ is a lossy label of $\tilde{pk}$. If $(pw, \cdot) \notin \mathcal{L}_H$, then $\mathcal{A}$ did not ever query $H(pw)$, so $H(pw)$ is random to $\mathcal{A}$. With a similar argument as Case 1, we can replace c with a random one in a computationally indistinguishable way.

Case 2.3 implies $\text{IsLossy}(td, \tilde{pk}, r_H) = \text{IsLossy}(td, \tilde{pk}, r'_H) = 0$, which happens with negligible probability according to the property of *uniqueness of normal labels among polynomial-size set* (among the all the labels stored in $\mathcal{L}_H$).

Accounting for the above cases, we have $|\Pr[G_8 \Rightarrow 1] - \Pr[G_7 \Rightarrow 1]| \leq \text{negl}(\lambda)$.

Game G_9 (simulation of generating C in the second-round message). In G_9 , Sim is the same as in G_8 , except for Sim 's simulation of generating C in the second-round message (c, C) for client instances. We consider the same cases as in G_8 . In Case 1, Case 2.1 and the sub-case $pw' \neq pw$ in Case 2.2, Sim invokes $C \leftarrow \text{CEnc}(epk, 0; r)$ rather than $C \leftarrow \text{CEnc}(epk, pw|pk|c; H_1(m))$ as did in G_8 .

Note that r is random and independent of $pw|pk|c$. According to the CCA security of PKE and hybrid arguments over the (at most) ℓ ciphertexts, we have

$$|\Pr[G_9 \Rightarrow 1] - \Pr[G_8 \Rightarrow 1]| \leq \ell \cdot \text{Adv}_{\text{PKE}}^{\text{cca}}(\mathcal{B}_{\text{PKE}}) \leq \text{negl}(\lambda).$$

Game G_{10} (simulation of dealing with the third-round message $\tilde{\sigma}$ for Server Instances in Case of Active Attacks). In G_{10} , Sim is the same as in G_9 , except for Sim 's simulation of generating sKey upon receiving the third-round message $\tilde{\sigma}$. Consider the same cases defined in G_9 (also G_8). In Case 1, Case 2.1 and the sub-case $pw' \neq pw$ in Case 2.2, Sim sets $\text{sKey} := \perp$ directly in G_{10} regardless of whether $\sigma = \tilde{\sigma}$ or not.

G_{10} is the same as G_9 except that $\sigma = \tilde{\sigma}$ happens in these cases in G_9 . However, σ is uniformly chosen and independent of other variables, and hence $\mathcal{A}$ can present a correct guess of σ with negligible probability. So we have

$$|\Pr[G_{10} \Rightarrow 1] - \Pr[G_9 \Rightarrow 1]| \leq \text{negl}(\lambda).$$

Now Sim does not use pw anymore except for the comparison $pw' \stackrel{?}{=} pw$ in Case 2.2 for server instances and in the rejection rule $(\star)$ for client instances.

Game G_{11} (Integration of Sim with $\mathcal{F}_{\text{pake}}$). G_{11} is the same as G_{10} , except that Sim accesses $\mathcal{F}_{\text{pake}}$ by issuing $\text{Testpw}(pw')$ to decide $pw' \stackrel{?}{=} pw$ for Case 2.2 and $(\star)$. (The detail of G_{11} and analysis is shown in the full version [25].) Note that $\text{Testpw}(pw')$ and $pw' \stackrel{?}{=} pw$ has the same functionality, so

$$\Pr[G_{11} \Rightarrow 1] = \Pr[G_{10} \Rightarrow 1].$$

Now that Sim completely gets rid of pw in the simulation, it finally arrives at its final form and G_{11} is exactly $\mathbf{Ideal}_{\mathcal{Z}, \text{Sim}}$.

Finally, by combining all the statements across G_0 - G_{11} , we know that

$$|\Pr[\mathbf{Real}_{\mathcal{Z}, \mathcal{A}}] - \Pr[\mathbf{Ideal}_{\mathcal{Z}, \text{Sim}}]| \leq \text{negl}(\lambda).$$

□

Remark 2. For our UC-secure PAKE construction in ROM, it is possible for us to remove the CCA-secure encryption C in the second-round message, resulting in a more efficient PAKE scheme without affecting its UC security. Note that C was originally used to extract password pw in the proof. Benefiting from the RO model, now the simulator can accomplish the extraction of pw without C using

a new strategy: the simulator checks if there exists a query $(pw, H(pw))$ and a query $(m, H_1(m))$ s.t. $c = \text{LPKE.Enc}(pk, H(pw), m; H_1(m))$ so as the password pw and the encrypted message m can be extracted from the ciphertext c . However, for our UC-secure PAKE construction in QROM, the new strategy does not work and we still need the CCA encryption of C for the extraction.

4 PAKE from Extractable LPKE in QROM

When considering post-quantum security in QROM, we have to consider a quantum adversary $\mathcal{A}$ making quantum superposition access to the random oracle. The proving technique of keeping hash query lists and searching for all queries in the lists in ROM does not apply any more, since the no-cloning principle makes impossible to maintain a query list.

In order to achieve UC security in QROM, we have to adjust the four hash functions H, H_1, H_2, H_3 in our PAKE construction to avoid keeping hash lists for them in the proof.

- In PAKE^{RO} , hash function H_1 helps LPKE to achieve CCA security as a KEM. According to [20], the FO-transformation works well in QROM.
- In PAKE^{RO} , the hash list of H_1 is also used for decryptions of c without secret key sk . To eliminate hash list of H_1 , we resort to the online-extractability technique due to Don et al. [20]. With this technique, we can construct a simulator $\mathcal{S} = (\mathcal{S}.RO, \mathcal{S}.E)$ who not only extracts the decrypting result m from $c = \text{LEnc}(pk, b, m; H_1(m)) = f_{pk,b}(m, H_1(m))$ with $\mathcal{S}.E$, but also simulates H_1 with $\mathcal{S}.RO$ in QROM. Note that the unpredictability of ciphertexts and the correctness of LPKE ensure not too many y 's satisfy $f_{pk,b}(m, y) = c$, which is a necessary condition for online-extractability to apply.
- In PAKE^{RO} , hash function H_2 and H_3 are used as PRF. Applying the O2H lemma enables us to prove the pseudo-randomness of $r|\sigma|k := H_2(m)$ and $\text{sKey} := H_3(k|sid)$ in QROM, when m and k are uniform.
- In PAKE^{RO} , hash list of H is used to extract password pw from the first-round message pk in the security proof in ROM. However, when formalizing H as a quantum accessible random oracle in $pk = \text{LKeyGen}(H(pw); r)$, an obstacle comes in the way: The O2H lemma does not apply because $\text{LKeyGen}(H(pw); r)$ is not in the form of $H(k|\cdot)$. The online-extractability does not apply either, since the simulator is not able to obtain the randomness r sampled by $\mathcal{A}$ and hence cannot determine the function f to be extracted. To eliminate the obstacle, we discard hash function H and replace the building block LPKE with an “extractable” one named eLPKE. To accomplish password extraction from pk in QROM, eLPKE directly uses pw as its label and uses an extra algorithm **Extract** to extract pw from pk with the help of trapdoor.

In Subsect. 4.1, we introduce the concept of extractable LPKE. In Subsect. 4.2, we show how to construct extractable LPKE from LPKE. In Subsect. 4.3, we show the generic construction of PAKE and prove its UC security in QROM.

4.1 Definition of Extractable LPKE (eLPKE)

When augmenting an extracting algorithm to a basic LPKE, we obtain *extractable LPKE* (eLPKE in short). An eLPKE scheme additionally requires *lossiness of random public keys* and *extractability of the unique normal label*, besides the properties of ①②⑤⑥ as per basic LPKE (see Sect. 3.1).

Definition 6. An *extractable LPKE scheme* $\text{eLPKE} = (\text{eLSetup}, \text{eLKeyGen}, \text{eLEnc}, \text{eLDec}, \text{elsLossy}, \text{Extract})$ consists of six algorithms, where $(\text{eLSetup}, \text{eLKeyGen}, \text{eLEnc}, \text{eLDec}, \text{elsLossy})$ are defined in the same way as the five algorithms $(\text{LSetup}, \text{LKeyGen}, \text{LEnc}, \text{LDec}, \text{lsLossy})$ in LPKE (cf. Def. 4). Algorithm *Extract* is defined below.

- $\text{Extract}(td, pk)$: The extracting algorithm takes as input a trapdoor td and a public key pk , and outputs a label b .

Correctness of eLPKE. It has the same correctness requirement as LPKE.

An extractable LPKE scheme eLPKE should satisfy the following properties and security requirements.

- ① **Pseudorandomness of Public Keys.** Same as ① of LPKE. (cf. Def. 4)
- ② **Random Ciphertexts under Lossy Labels.** Same as ② of LPKE.
- ③ **Extractability of the Unique Normal Label.** For every $\mathcal{A}$, it holds that

$$\Pr \left[\begin{array}{l} (crs, td) \leftarrow \text{eLSetup} \\ pk \leftarrow \mathcal{A}(crs); b \leftarrow \text{Extract}(td, pk) : \exists b' \neq b, \text{elsLossy}(td, pk, b') = 0 \end{array} \right] \leq \text{negl}(\lambda).$$

- ④ **Lossiness of Random Public Keys.** For every adversary $\mathcal{A}$, it holds that

$$\Pr \left[\begin{array}{l} (crs, td) \leftarrow \text{eLSetup} \\ b \leftarrow \mathcal{A}(crs), pk \leftarrow_{\$} \mathcal{PK} : \text{elsLossy}(td, pk, b) = 0 \end{array} \right] \leq \text{negl}(\lambda).$$

- ⑤ **Ciphertext Unpredictability under Normal Labels.** Same as ⑤ of LPKE. (cf. Def. 4)
- ⑥ **CPA Security under Normal Label.** Same as ⑥ of LPKE. (cf. Def. 4)

Remark 3. Property ③ means that for adversary's choice of pk , either there exists no normal label for pk or $\text{Extract}(td, pk)$ can extract a unique normal label for pk . In fact, Property ③ along with correctness of eLPKE imply that if $(pk, sk) \leftarrow \text{eLKeyGen}(b)$ then $\text{Extract}(td, pk)$ outputs the unique normal label b of pk .

4.2 Construction of eLPKE from LPKE⁺

Given a basic LPKE = (LSetup, LKeyGen, LEnc, LDec, lsLossy) with message space $\{0, 1\}^\lambda$ and label space $\mathcal{T}$, we construct an extractable LPKE scheme eLPKE with message space $\{0, 1\}^\lambda$ and label space $\{0, 1\}^\lambda$. See Fig. 7 for eLPKE construction.

We will prove the property of eLPKE if the underlying basic LPKE not only satisfies property ①②③⑤⑥ (cf. Definition 4) but also two additional properties ④ (cf. Def. 6) and ⑦, where ⑦ is defined below.

$\text{eLSetup}(1^\lambda) :$ $(\text{pp}', \text{td}') \leftarrow \text{LSetup}$ For $i := 1$ to λ : $v_i^0, v_i^1 \leftarrow \mathcal{T}$ Return $(\text{pp} := (\text{pp}', \{v_i^0, v_i^1\}_{i \in [\lambda]}), \text{td} := \text{td}')$ $\text{eLKeyGen}(\mathbf{b} = b_1 \dots b_\lambda \in \{0, 1\}^\lambda) :$ For $i := 1$ to λ : $(pk_i, sk_i) \leftarrow \text{LKeyGen}(v_i^{b_i})$ Return $\mathbf{pk} := (pk_1, \dots, pk_\lambda), \mathbf{sk} := (sk_1, \dots, sk_\lambda)$ $\text{eLEnc}(\mathbf{pk}, \mathbf{b} = b_1 \dots b_\lambda \in \{0, 1\}^\lambda, m \in \{0, 1\}^\lambda) :$ Parse $\mathbf{pk} := (pk_1, \dots, pk_\lambda)$ For $i := 1$ to $\lambda - 1$: $z_i \leftarrow \{0, 1\}^\lambda$ $z_\lambda := m \oplus z_1 \oplus \dots \oplus z_{\lambda-1}$ For $i := 1$ to λ : $c_i \leftarrow \text{LEnc}(pk_i, v_i^{b_i}, z_i)$ Return $\mathbf{c} := (c_1, \dots, c_\lambda)$	$\text{eLDec}(\mathbf{sk} = (sk_1, \dots, sk_\lambda), \mathbf{c} = (c_1, \dots, c_\lambda)) :$ For $i := 1$ to λ : $z_i \leftarrow \text{LDec}(sk_i, c_i)$ Return $m := z_1 \oplus \dots \oplus z_\lambda$ $\text{elsLossy}(\text{td}, \mathbf{pk}, \mathbf{b} = b_1 \dots b_\lambda \in \{0, 1\}^\lambda) :$ Parse $\mathbf{pk} = (pk_1, \dots, pk_\lambda)$ For $i := 1$ to λ : If $\text{IsLossy}(\text{td}, pk_i, v_i^{b_i}) = 1$: Return 1 Return 0 $\text{Extract}(\text{td}, \mathbf{pk} = (pk_1, \dots, pk_\lambda)) :$ For $i := 1$ to λ : If $\text{IsLossy}(\text{td}, pk_i, v_i^0) = 0$: $b_i := 0$ Else: $b_i := 1$ Return $\mathbf{b} := b_1 \dots b_\lambda$
---	---

Fig. 7. Construction of eLPKE from LPKE⁺.

⑦ **Ciphertext Randomness in case of Random Messages for LPKE.** For all $(\text{pp}, \text{td}) \leftarrow \text{LSetup}(1^\lambda)$, every (possibly malformed) public key $\mathbf{pk}$, every label $\mathbf{b} \in \mathcal{T}$ (no matter whether $\text{IsLossy}(\text{td}, \mathbf{pk}, \mathbf{b}) = 1$ or not), it holds that $\mathbf{c} \approx_s \$$, where $m \leftarrow \$$, $\{0, 1\}^\lambda$ and $\mathbf{c} \leftarrow \text{LEnc}(\mathbf{pk}, \mathbf{b}, m)$.

We designate such a LPKE as LPKE⁺ if it satisfies ①②③④⑤⑥⑦.

Theorem 2. For the construction of eLPKE in Fig. 7, if the underlying LPKE is a LPKE⁺ scheme, i.e., it satisfies ①②③④⑤⑥⑦, then the resulting eLPKE scheme has the properties of ①②③④⑤⑥ and supports label space $\{0, 1\}^\lambda$.

Proof. For eLPKE, its properties of ①④⑤⑥ follow directly from ①④⑤⑥ of the underlying LPKE⁺. Now we prove ②③ for eLPKE.

② **Random Ciphertexts under Lossy Labels:** We aim to show $(\text{pp}, \mathbf{pk}, \mathbf{b}, m, \mathbf{c}) \approx_s (\text{pp}, \mathbf{pk}, \mathbf{b}, m, \$)$, where $(\text{pp}, \text{td}) \leftarrow \text{eLSetup}$, $(\mathbf{pk}, \mathbf{b}) \leftarrow \mathcal{A}(\text{pp})$ s.t. $\text{elsLossy}(\text{td}, \mathbf{pk}, \mathbf{b}) = 1$, $m \leftarrow \$$, $\{0, 1\}^\lambda$, $z_i \leftarrow \$$, $\{0, 1\}^\lambda$ for $i \in [\lambda - 1]$, $z_\lambda := m \oplus z_1 \oplus \dots \oplus z_{\lambda-1}$, $c_i \leftarrow \text{LEnc}(pk_i, v_i^{b_i}, z_i)$ for $i \in [\lambda]$ and $\mathbf{c} = c_1 | \dots | c_\lambda$. Given $\text{elsLossy}(\text{td}, \mathbf{pk}) = 1$, there must exist a position j such that $\text{IsLossy}(\text{td}, pk_j, v_j^{b_j}) = 1$. Then we can replace c_j in $\mathbf{c}$ with a random $c_j \leftarrow \mathcal{CT}$ by the property of “② random ciphertexts under lossy labels” of the underlying LPKE⁺ scheme.

Now m and $\{z_i\}_{i \in [\lambda]}$ can be sampled in an equivalent way: $m \leftarrow \$$, $\{0, 1\}^\lambda$, $z_i \leftarrow \$$, $\{0, 1\}^\lambda$ for $i \in [\lambda] \setminus \{j\}$, and set $z_j := m \oplus \bigoplus_{i=1, i \neq j}^\lambda z_i$. Note that plaintexts $\{z_i\}_{i \in [\lambda] \setminus \{j\}}$ encrypted in $\{c_i\}_{i \in [\lambda] \setminus \{j\}}$ are independent of message m . Therefore, by “⑦ ciphertext randomness in case of random messages” of underlying LPKE⁺ scheme, we can replace c_i with a random $c_i \leftarrow \mathcal{CT}$ for each $i \in [\lambda] \setminus \{j\}$. Together with the random ciphertext c_j , we arrive at the right side in a statistical indistinguishable way.

③ **Extractability of the Unique Normal Label:** Suppose toward contradiction, there is a $\mathbf{b}'$ such that $\mathbf{b}' \neq \mathbf{b}$ and $\text{elsLossy}(\text{td}, \mathbf{pk}, \mathbf{b}') = 0$, where $(\text{pp}, \text{td}) \leftarrow \text{eLSetup}$, $\mathbf{pk} \leftarrow \mathcal{A}(\text{pp})$, $\mathbf{b} \leftarrow \text{Extract}(\text{td}, \mathbf{pk})$.

Given $\mathbf{b}' \neq \mathbf{b}$, there must exist a position $j \in [\lambda]$ s.t. $b'_j \neq b_j$. Given $\text{elsLossy}(\text{td}, \mathbf{b}', \mathbf{pk}) = 0$, it holds that $\text{IsLossy}(\text{td}, pk_i, v_i^{b'_i}) = 0$ for $i \in [\lambda]$. Now

we have $b'_j \neq b_j$ and $\text{IsLossy}(td, pk_j, v_j^{b'_j}) = 0$. According to the specification of $\mathbf{b} \leftarrow \text{Extract}(td, \mathbf{pk})$, we know that $b_j = 0$ if and only if $\text{IsLossy}(td, pk_j, v_j^0) = 0$. We consider two cases according to the value of b_j .

Case 1: $b_j = 0$ (so $b'_j = 1$). In this case, we have $\text{IsLossy}(td, pk_j, v_j^0) = \text{IsLossy}(td, pk_j, v_j^1) = 0$, which contradicts to property ③ of the LPKE⁺.

Case 2: $b_j = 1$ (so $b'_j = 0$). In this case, we have $\text{IsLossy}(td, pk_j, v_j^1) \neq 0$ which contradicts to the fact that $\text{IsLossy}(td, pk_j, v_j^{b'_j}) = \text{IsLossy}(td, pk_j, v_j^0) = 0$. $\square$

Remark 4. With the FO-transformation, we can construct a KEM scheme KEM' from eLPKE. Note that Lemma 4 and Lemma 5 remain applicable to eLPKE since eLPKE has properties of ②⑤⑥. Therefore, KEM' has both CCA-security and ciphertext pseudo-randomness under lossy labels.

4.3 Construction of PAKE from eLPKE in QROM

Replacing the building block of basic LPKE with eLPKE, we obtain the generic construction of PAKE from eLPKE. The resulting PAKE scheme PAKE^{QRO} is shown in Fig. 6 and its UC security proof in QROM is shown in Theorem 3.

Theorem 3. *If eLPKE is an extractable LPKE scheme, PKE is a CCA-secure PKE, and H_1, H_2, H_3 are quantum-accessible random oracles, then scheme PAKE^{QRO} in Fig. 6 securely emulates $\mathcal{F}_{pake}$, hence achieving UC security in QROM.*

The proof outline is similar to that of Theorem 1, where $G'_1, G'_2, G'_5, G'_7, G'_9$ - G'_{11} are the same as G_1, G_2, G_5, G_7, G_9 - G_{11} . Note that the simulator will not keep the hash lists. The differences lie in G'_3, G'_4, G'_6, G'_8 , for which we give a brief overview.

G'_3 & G'_8 . We do not maintain hash list for $H_2(\cdot)$ to compute $r|\sigma|k$ as did in G_3 & G_8 . Instead, we make use of Lemma 4 & Lemma 5 to argue $r|\sigma|k := H_2(m)$ is pseudo-random.

G'_4 . We do not maintain hash list for $H_3(\cdot)$ to compute $sKey$ as did in G_4 . Instead, we make use of O2H Lemma [6] to argue $sKey := H_3(k|sid)$ is pseudorandom.

G'_6 . For client instances, we have to eliminate the usage of sk of eLPKE so that the first round message pk can be replaced with a random one in G'_7 . We do not use the hash list for $H_1(\cdot)$, as did in G_6 . Instead we resort to the online-extractability technique and make use of the corresponding simulator $\mathcal{S} = (\mathcal{S}.RO, \mathcal{S}.E)$ to simulates random oracle $H_1(\cdot)$ and extract the decryption result of $\text{eLDec}(sk, c)$.

G'_8 . For server instances receiving the first-round message pk in an active attack, we do not maintain the hash list for $H(\cdot)$ and search the list to find the correct password pw (if exists). Instead we use $\text{Extract}(td, pk)$ to extract pw .

The full description of the proof is shown in the full version [25].

5 Instantiations

In our generic PAKE constructions, one building block is a CCA-secure PKE scheme, which can be easily obtained from CPA-secure PKE via FO-transformation in (quantum) ROM [33]. For example, by applying FO-transformation to the Regev PKE [30] or ElGamal-like PKE from GA-DDH (Sect. 7.1 in [12]), we can obtain CCA-secure PKE from the LWE or GA-DDH assumption. Therefore, to obtain specific PAKE schemes, we only consider the instantiations of LPKE.

In Subsect. 5.1 and 5.2, we give the instantiations of LPKE and LPKE^+ , from LWE and GA-DDH respectively. The instantiations of CCA-secure PKE and LPKE yield four specific post-quantum UC-secure PAKE schemes, two in ROM and the other two in QROM, as shown in Subsect. 5.3.

5.1 LPKE and LPKE^+ Schemes from LWE

We present LWE-based LPKE scheme LPKE_{lwe} and LPKE^+ scheme $\text{LPKE}'_{\text{lwe}}$. Before presenting our scheme LPKE_{lwe} , we recall some technical tools including an important algorithm called `IsMessy` introduced in [21].

- **Statistical distance** $\delta_{q,r}(\mathbf{A}, \mathbf{x})$. Given $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{x} \in \mathbb{Z}_q^m$, define

$$\delta_{q,r}(\mathbf{A}, \mathbf{x}) := \Delta((\mathbf{A}\mathbf{e}, \mathbf{x}^T \mathbf{e}), (\mathbf{u}, u)), \quad (1)$$

where $\mathbf{e} \leftarrow D_{\mathbb{Z}^m, r}$, $\mathbf{u} \leftarrow_s \mathbb{Z}_q^n$, $u \leftarrow_s \mathbb{Z}_q$ and Δ is the statistical distance.

- **Algorithm** `IsMessy`($\mathbf{T}_{\mathbf{A}}, \mathbf{A}, \mathbf{x}$). It takes as input a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{A}$'s trapdoor $\mathbf{T}_{\mathbf{A}}$, and a vector $\mathbf{x} \in \mathbb{Z}_q^m$ and outputs “messy” or “not sure”.

Lemma 6 (Proposition 7.8 in [28]). *Let $m \geq 2(n+1)\log q$ and let $r \geq \sqrt{qm} \cdot \log^2 m$. Suppose that $(\mathbf{A}, \mathbf{T}_{\mathbf{A}})$ are generated by `TrapGen` (c.f. Lemma 1). Then the PPT algorithm `IsMessy`($\mathbf{T}_{\mathbf{A}}, \mathbf{A}, \mathbf{x}$) satisfying the following statements.*

- With overwhelming probability over the choice of $\mathbf{A}, \mathbf{T}_{\mathbf{A}}$, for all but an at most $(1/2\sqrt{q})^m$ fraction of vectors $\mathbf{x} \in \mathbb{Z}_q^m$, `IsMessy`($\mathbf{T}_{\mathbf{A}}, \mathbf{A}, \mathbf{x}$) outputs “messy” with overwhelming probability (over its own randomness).*
- There exists $\epsilon(\lambda) = \text{negl}(\lambda)$ such that if $\delta_{q,r}(\mathbf{A}, \mathbf{x}) > 2\epsilon(\lambda)$, then `IsMessy`($\mathbf{T}_{\mathbf{A}}, \mathbf{A}, \mathbf{x}$) = “not sure”, with overwhelming probability (over its own randomness). (In other words, if `IsMessy`($\mathbf{T}_{\mathbf{A}}, \mathbf{A}, \mathbf{x}$) = “messy”, then $\delta_{q,r}(\mathbf{A}, \mathbf{x}) = \text{negl}$ with overwhelming probability.)*

Our basic LPKE scheme LPKE_{lwe} is described as follows.

LSetup (1^λ) :	LDec ($sk = \mathbf{s}, ct = (\mathbf{u}_1 \dots \mathbf{u}_\lambda, c_1 \dots c_\lambda)$) :
$(\mathbf{A}, \mathbf{T}_\mathbf{A}) \leftarrow \text{TrapGen}(1^\lambda)$	For $i := 1$ to λ :
Return $(pk := \mathbf{A}, td := \mathbf{T}_\mathbf{A})$	$d_i := c_i - \mathbf{s}^T \mathbf{u}_i$
LKeyGen ($\mathbf{v} \in \mathbb{Z}_q^m$) :	If $d_i \in [q/4, 3q/4]$: $m_i := 1$
$\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow D_{\mathbb{Z}_q^m, \sigma}$	Else $m_i := 0$
Return $(pk := \mathbf{A}^T \mathbf{s} + \mathbf{e} - \mathbf{v}, sk := \mathbf{s})$	Return $m := m_1 \dots m_\lambda$
LEnc ($pk, \mathbf{v}, m \in \{0, 1\}^\lambda$) :	IsLossy ($td = \mathbf{T}_\mathbf{A}, pk, \mathbf{v}$) :
$\mathbf{p} := pk + \mathbf{v}$	If IsMessy ($\mathbf{T}_\mathbf{A}, \mathbf{A}, pk + \mathbf{v}$) = “messy”:
For $i := 1$ to λ :	Return 1
$\mathbf{e}_i \leftarrow D_{\mathbb{Z}_q^m, r}, \mathbf{u}_i := \mathbf{A} \mathbf{e}_i$	Else: Return 0
$c_i := \mathbf{p}^T \mathbf{e}_i + m_i \cdot \frac{q}{2}$	
Return $ct := (\mathbf{u}_1 \dots \mathbf{u}_\lambda, c_1 \dots c_\lambda)$	

Fig. 8. The basic LPKE scheme LPKE_{lwe} .

Theorem 4. If $q > 4r\sigma m \log^2 n$, $m > 2(n+1) \log q$, and $r > \sqrt{qm} \log^2 m$, then LPKE_{lwe} is a basic LPKE scheme satisfying properties ①–⑥ based on $\text{LWE}_{n,q,m,D_{\mathbb{Z}_q^m,\sigma}}$ assumption.

Proof. We prove that LPKE_{lwe} has correctness and the corresponding properties. By the tail bound, $\|\mathbf{e}\| \leq \omega(\sqrt{\log n}) \cdot \sigma\sqrt{m}$ and $\|\mathbf{e}_i\| \leq \omega(\sqrt{\log n}) \cdot r\sqrt{m}$. The correctness follows from $|\mathbf{e}^T \mathbf{e}_i| \leq \|\mathbf{e}\| \|\mathbf{e}_i\| \leq r\sigma m \cdot \omega(\log n) < q/4$.

① **Pseudorandomness of Public Key:** According to Lemma 1, the matrix $\mathbf{A}$ outputted from **TrapGen** is statistically close to a uniform distribution. Then *pseudorandomness of public key* follows from the LWE assumption.

② **Random Ciphertext under Lossy Labels:** By statement (b) of Lemma 6 and (1), we know $(\mathbf{A} \mathbf{e}_i, \mathbf{p}^T \mathbf{e}_i) \approx_s (\$, \$)$ and independent of $(\mathbf{A}, pk, \mathbf{v})$ for $i \in [\lambda]$. So we have $(\mathbf{A}, pk, \mathbf{v}, m, \{\mathbf{A} \mathbf{e}_i, \mathbf{p}^T \mathbf{e}_i + m_i \cdot \frac{q}{2}\}_{i \in [\lambda]}) \approx_s (\mathbf{A}, pk, \mathbf{v}, m, \{\$, \$\}_{i \in [\lambda]})$.

③ **Uniqueness of Normal Labels among Polynomial-Size Set:** According to statement (a) of Lemma 6, we know that there are $1 - \text{negl}(\lambda)$ fraction of matrices $\mathbf{A}$ (and $\mathbf{T}_\mathbf{A}$) such that $\Pr[\mathbf{x} \leftarrow \mathbb{Z}_q^m : \text{IsMessy}(\mathbf{T}_\mathbf{A}, \mathbf{A}, \mathbf{x}) = \text{“messy”}] \geq 1 - \text{negl}(\lambda)$. Let $\mathbf{A}$ (and $\mathbf{T}_\mathbf{A}$) be such a fixed matrix, and define set $\mathcal{S} := \{\mathbf{x} \in \mathbb{Z}_q^m \mid \text{IsMessy}(\mathbf{T}_\mathbf{A}, \mathbf{A}, \mathbf{x}) = \text{“messy”}\}$. Then $\Pr[\mathbf{x} \leftarrow \mathbb{Z}_q^m : \mathbf{x} \notin \mathcal{S}] \leq (1/2\sqrt{q})^m$.

Further fixing a public key $pk = \mathbf{p} \in \mathbb{Z}_q^m$, we have

$$\Pr[\mathbf{x}_0, \mathbf{x}_1 \leftarrow \mathbb{Z}_q^m : \mathbf{p} + \mathbf{x}_0 \notin \mathcal{S} \wedge \mathbf{p} + \mathbf{x}_1 \notin \mathcal{S}] = (\Pr[\mathbf{x} \leftarrow \mathbb{Z}_q^m : \mathbf{x} \notin \mathcal{S}])^2 \leq (1/4q)^m.$$

By a union bound over $Q(Q-1)/2 (\leq Q^2)$ possible pairs of $(\mathbf{x}_i, \mathbf{x}_j)$, we have

$$\Pr[\mathbf{x}_1, \dots, \mathbf{x}_Q \leftarrow \mathbb{Z}_q^m : \exists i \neq j, i, j \in [Q], \mathbf{p} + \mathbf{x}_i \notin \mathcal{S} \wedge \mathbf{p} + \mathbf{x}_j \notin \mathcal{S}] \leq Q^2 \cdot (1/4q)^m.$$

If $\mathbf{x}_1, \dots, \mathbf{x}_Q \leftarrow \mathbb{Z}_q^m$, then with a union bound over all q^m possible $pk = \mathbf{p} \in \mathbb{Z}_q^m$, we have $\Pr[\exists \mathbf{p} \in \mathbb{Z}_q^m, \exists i \neq j, \mathbf{p} + \mathbf{x}_i \notin \mathcal{S} \wedge \mathbf{p} + \mathbf{x}_j \notin \mathcal{S}] \leq Q^2 \cdot (1/4)^m \leq \text{negl}(\lambda)$, so $\Pr[\exists \mathbf{p} \in \mathbb{Z}_q^m, \exists i \neq j, \text{IsLossy}(\mathbf{T}_\mathbf{A}, \mathbf{p}, \mathbf{x}_i) = 0 \wedge \text{IsLossy}(\mathbf{T}_\mathbf{A}, \mathbf{p}, \mathbf{x}_j) = 0] \leq \text{negl}(\lambda)$.

- ④ **Lossiness of Random Labels:** It follows directly from (b) of Lemma 6.
- ⑤ **Ciphertext Unpredictability under Normal Labels:** According to Lemma 3, we know that $\mathbf{A}\mathbf{e}$ is statistically close to a uniform distribution, hence the probability that a ciphertext output from the encryption algorithm collides with a fixed ciphertext should be negligible.
- ⑥ **CPA Security under Normal Labels:** Indeed, the encryption scheme under a normal label is a variant of the Regev public key encryption scheme [30]. So it naturally inherits the CPA security from the Regev cryptosystem. $\square$

Remark 5. Our basic LPKE scheme is adapted from the LWE-based dual-mode PKE scheme [28] but with the following differences.

- **Different label space.** The dual-mode PKE [28] only supports a simple label space $\{0, 1\}$, while our LPKE_{lwe} has label space $\mathbb{Z}_q^m$, which is compatible to the hash value $H(pw)$ of password in the PAKE scheme.
- **Different syntax.** In [28], the CRS consists of two fixed vectors corresponding to label 0 and label 1 respectively. And there are two indistinguishable ways for generating CRS to determine the normal mode or the messy mode. In our LPKE_{lwe} , the CRS only has one mode and the public key together with a label determine the encryption is in a normal mode or a lossy mode.
- **Different security requirements.** Due to the different syntax and different applications, we also have different security requirements for LPKE_{lwe} .

Parameters. Set $n = \Theta(\lambda)$, $m = \Theta(\lambda \log \lambda)$, $q = \Theta(\lambda^5 \log^3 \lambda)$, $r = \Theta(\lambda^{3.5})$, and $\sigma = \Theta(\sqrt{\lambda})$. Such parameters satisfy the requirements in Theorem 4.

LPKE_{lwe} is not a LPKE^+ scheme since it does not satisfy “⑦ *ciphertext randomness in case of random messages*”. Recall that the ciphertext element $c_i = \mathbf{p}^T \mathbf{e}_i + m_i \cdot \frac{q}{2}$. We can consider $k_i := \mathbf{p}^T \mathbf{e}_i \in \mathbb{Z}_q$ as an ephemeral key used to hide the bit $m_i \in \{0, 1\}$. However, the uniformity of one bit $m_i \in \{0, 1\}$ is not sufficient to fully randomize $c_i \in \mathbb{Z}_q$. To solve this problem, we introduce the novel round function $R(\cdot)$ parameterized by T from [11] to result in $k_i := R(\mathbf{p}^T \mathbf{e}_i) \in \{0, 1\}$ so that $c_i := k_i \oplus m_i \in \{0, 1\}$. In this way, property ⑦ is achieved. However, CPA-security is lost since $R(\mathbf{p}^T \mathbf{e}_i)$ is not pseudo-random. To fix that, we add multiple $R(\mathbf{p}^T \mathbf{e}_{ij})$ to get $k_i := \bigoplus_{j \in [\lambda]} R(\mathbf{p}^T \mathbf{e}_{ij})$. By the LWE assumption and leftover hash lemma, $\mathbf{p}^T \mathbf{e}_{ij}$ can be replaced by a uniform element $\mathbf{u}_{ij}$. Then the rounding function R makes $R(\mathbf{u}_{ij})$ follow a Bernoulli distribution with parameter about $\frac{1}{3}$. By the piling-up lemma [1], $k_i := \bigoplus_{j \in [\lambda]} R(\mathbf{u}_{ij})$ is statistically to the uniform distribution, thus achieving CPA-security.

LSetup (1^λ) :	LDec ($sk = s, ct$) :
($\mathbf{A}, \mathbf{T}_\mathbf{A}$) $\leftarrow$ TrapGen($1^n, 1^m, q$)	Parse $ct = (\{\mathbf{c}_{ij}\}_{i,j \in [\lambda]}, \{\beta_i\}_{i \in [\lambda]})$.
Return ($pp := \mathbf{A}, td := \mathbf{T}_\mathbf{A}$)	For $i := 1$ to λ :
LKeyGen ($\mathbf{v} \in \mathbb{Z}_q^m$) :	$m_i := \bigoplus_{j \in [\lambda]} R(\mathbf{s}^T \mathbf{c}_{ij}) \oplus \beta_i$
$\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathbb{D}_{\mathbb{Z}^m, t}$	Return $m = m_1 \dots m_\lambda$
Return ($pk := \mathbf{A}^T \mathbf{s} + \mathbf{e} - \mathbf{v}, sk := \mathbf{s}$)	
LEnc ($pk, m = m_1 \dots m_\lambda \in \{0, 1\}^\lambda, \mathbf{v}$) :	IsLossy ($td = \mathbf{T}_\mathbf{A}, pk, \mathbf{v}$) :
$\mathbf{p} := pk + \mathbf{v}$	$\mathbf{p} := pk + \mathbf{v}$
For $i := 1$ to λ :	($\mathbf{s}, \mathbf{e}$) $\leftarrow$ Invert($\mathbf{T}_\mathbf{A}, \mathbf{A}, \mathbf{p}$)
For $j := 1$ to λ :	// See Lemma 2 for Invert
$\mathbf{e}_{ij} \leftarrow D_{\mathbb{Z}^m, r}, \mathbf{c}_{ij} := \mathbf{A} \mathbf{e}_{ij}$	If $\ \mathbf{e}\ \leq q/8\sqrt{m}$: Return 0
$\beta_i := (\bigoplus_{j \in [\lambda]} R(\mathbf{p}^T \mathbf{e}_{ij})) \oplus m_i$	Else: Return 1
Return $ct = (\{\mathbf{c}_{ij}\}_{i,j \in [\lambda]}, \{\beta_i\}_{i \in [\lambda]})$	

Fig. 9. LPKE⁺ scheme LPKE'_{lwe}.

Our LPKE⁺ scheme LPKE'_{lwe} is adapted from [11, 29] and shown in Fig. 9. The security of LPKE'_{lwe} shown in Theorem 5 is proved in the full version [25]

Theorem 5. *If $r > n \log n$, $trm/q = \text{negl}(\lambda)$, $T/q = \text{negl}(\lambda)$, $m = \Theta(n \log n)$, and $\frac{\sqrt{m}}{r}(\frac{nq}{T})^2 < \Theta(\sqrt{m})$, then LPKE'_{lwe} in Fig. 9 is a LPKE⁺ scheme based on the $\text{LWE}_{n,q,m,D_{\mathbb{Z}^m,t}}$ assumption.*

Parameters. Set $n = \Theta(\lambda)$, $q = 2^\lambda$, $m = \Theta(\lambda^2)$, $t = \tilde{O}(\lambda^{3/2})$, $T = 2^{\frac{2\lambda}{3}}$, $k = \Theta(\lambda)$, and $r = \Omega(\lambda^4 \cdot 2^{\frac{2\lambda}{3}})$. Such parameters satisfy the requirements in Theorem 5.

5.2 LPKE and LPKE⁺ Scheme from Group Actions

The second LPKE scheme LPKE_{ga} is based on restricted effective group actions (REGA). It is adapted from the construction of the dual-mode PKE from group actions in [5]. Our LPKE_{ga} vs. the dual-mode PKE from group actions in [5] is analogous to LPKE_{lwe} vs. the dual-mode PKE from LWE in [28] (See Remark 5).

Let $(\mathbb{G}, \mathcal{X}, \star)$ be a REGA, $\mathcal{H} = \{\mathbf{H} : \mathcal{X}^\ell \rightarrow \{0, 1\}\}$ be a family of pairwise independent universal hash functions. Scheme LPKE_{ga} is described in Fig. 10.

LSetup (1^λ) :	LKeyGen ($pp, g \in \mathbb{G}$) :
$x \leftarrow \mathcal{X}, t \leftarrow \mathbb{G}, \bar{x} \leftarrow t \star x, \mathbf{H} \leftarrow \mathcal{H}$	$s \leftarrow \mathbb{G}$
Return ($pp := (x, \bar{x}, \mathbf{H}), td := t$)	Return ($pk := (s \star x, (g \cdot s) \star \bar{x}), sk := s$)
LEnc ($pk = (y, \bar{y}), g \in \mathbb{G}, m = m_1 \dots m_\lambda \in \{0, 1\}^\lambda$) :	LDec ($sk = s, ct$) :
For $i := 1$ to λ :	Parse $ct = (c_1, b_1), \dots, (c_\lambda, b_\lambda)$
$\mathbf{r}_i = (r_{i1}, \dots, r_{i\ell}) \leftarrow \mathbb{G}^\ell$	For $i := 1$ to λ :
$\mathbf{b}_i = (b_{i1}, \dots, b_{i\ell}) \leftarrow \{0, 1\}^\ell$	$\mathbf{z}_i := (s \star \mathbf{c}_i) = (s \star c_{i1}, \dots, s \star c_{i\ell})$
For $j := 1$ to ℓ :	$m_i := b_i \oplus \mathbf{H}(\mathbf{z}_i)$
If $b_{ij} = 0$: $c_{ij} := r_{ij} \star \bar{x}, c'_{ij} := r_{ij} \star (g^{-1} \star \bar{y})$	Return $m := m_1 \dots m_\lambda$
Else $c_{ij} := r_{ij} \star x, c'_{ij} := r_{ij} \star y$	
$\mathbf{c}_i := (c_{i1}, \dots, c_{i\ell}) \in \mathcal{X}^\ell, \mathbf{c}'_i := (c'_{i1}, \dots, c'_{i\ell}) \in \mathcal{X}^\ell$	IsLossy ($td = t \in \mathbb{G}, pk = (y, \bar{y}, g \in \mathbb{G})$) :
$ct_i := (\mathbf{c}_i, \mathbf{H}(\mathbf{c}'_i) \oplus m_i)$	If $\bar{y} = (t \cdot g) \star y$: Return 0
Return $ct := (ct_1, \dots, ct_\lambda)$	Else: Return 1

Fig. 10. LPKE and LPKE⁺ scheme LPKE_{ga}.

Theorem 6. *If $(\mathbb{G}, \mathcal{X}, \star)$ is a REGA, then LPKE_{ga} in Fig. 10 is both a LPKE scheme and a LPKE^+ scheme based on GA-DDH assumption.*

Proof. The proofs are similar to [5], so we only provide a concise overview.

① **Pseudorandomness of Public Keys:** We need to show that

$$(x, t \star x, s \star x, (g \cdot s \cdot t) \star x) \approx_c (x, t \star x, u_1, u_2) \quad (2)$$

for all $g \in \mathbb{G}$ provided by adversary $\mathcal{A}$, where $s, t \leftarrow_s \mathbb{G}$ and $u_1, u_2 \leftarrow_s \mathcal{X}$. The GA-DDH assumption requires $(x, s \star x, t \star x, (s \cdot t) \star x) \approx_c (x, s \star x, t \star x, z \star x)$, where $s, t, z \leftarrow_s \mathbb{G}$. Let g act on the last term, we have $(x, s \star x, t \star x, g \star ((s \cdot t) \star x)) \approx_c (x, s \star x, t \star x, g \star (z \star x))$. Note that $g \star ((s \cdot t) \star x) = (g \cdot s \cdot t) \star x$, and the uniformity and independence of x and z guarantees that $s \star x$ and $g \star (z \star x)$ are uniformly and independently distributed. So we obtain (2).

② **Random Ciphertexts under Lossy Labels:** Recall that $\text{pp} = (x, \bar{x} = t \star x, H), pk = (y, \bar{y})$. Given a lossy label $g \in \mathbb{G}$ from $\mathcal{A}$, we have $\text{lsLossy}(td, pk, g) = 1$, i.e., $\bar{y} \neq (t \cdot g) \star y$. When writing $\bar{y} = (t \cdot g') \star y$, we have $g \neq g'$. Now the encryption scheme using label g becomes

$$c_j = \begin{cases} (r_j \cdot t) \star x & \text{if } b_j = 0 \\ r_j \star x & \text{if } b_j = 1 \end{cases}, \quad c'_j = \begin{cases} (r_j \cdot (g^{-1} \cdot g') \cdot t) \star y & \text{if } b_j = 0 \\ r_j \star y & \text{if } b_j = 1 \end{cases}.$$

Conditioned on pp, pk, g, c_j , bit b_j is perfectly hidden in c_j thanks to the randomness of r_j . Suppose $y = s \star x$. Then given $(x, t \star x, y, (t \cdot g') \star y, g, m, c_j)$, we know either $c'_j = s \star (g^{-1} \cdot g') \star c_j$ in case of $b_j = 0$ or $c'_j = s \star c_j$ in case of $b_j = 1$. Now that $g \neq g'$, so c'_j has one bit entropy and hence $\mathbf{c}'$ has ℓ bits entropy.

As a result, the vector $\mathbf{c}' = (c'_1, \dots, c'_\ell)$ has ℓ bit entropy and by the leftover hash lemma, $H(\mathbf{c}')$ is close to uniform distribution, thus hiding m_i statistically.

③ **Uniqueness of Normal Labels among Polynomial-Size Set:** By the regularity of the group action, we have

$$\begin{aligned} & \Pr \left[\begin{array}{c} x \leftarrow_s \mathcal{X}, t \leftarrow_s \mathbb{G}, \bar{x} := t \star x \\ g_1, \dots, g_Q \leftarrow_s \mathbb{G} \end{array} : \begin{array}{c} \exists (y, \bar{y}) \in \mathcal{X} \times \mathcal{X}, i \neq j \\ \bar{y} = (t \cdot g_i) \star y \wedge \bar{y} = (t \cdot g_j) \star y \end{array} \right] \\ &= \Pr [g_1, \dots, g_Q \leftarrow_s \mathbb{G} : \exists i \neq j, g_i = g_j] \leq Q^2 / |\mathbb{G}| = \text{negl}(\lambda). \end{aligned}$$

④&④ **Lossiness of Random Labels/Random Public Keys:** By the regularity of the group action, we have $\Pr [g \leftarrow_s \mathbb{G} : \bar{y} = (t \cdot g) \star y] \leq 1/|\mathbb{G}| = \text{negl}(\lambda)$ for any fixed $y, \bar{y} \in \mathcal{X}$ and $t \in G$, and prove ④. For any fixed $g \in \mathbb{G}$ and $t \in \mathbb{G}$, we have $\Pr [y, \bar{y} \leftarrow_s \mathcal{X} : \bar{y} = (t \cdot g) \star y] \leq 1/|\mathcal{X}| = \text{negl}(\lambda)$ that proves ④.

⑤ **Ciphertext Unpredictability under Normal Labels:** For any fixed pp, pk, b, m, c , by the regularity of the group action, we have

$$\Pr [r \leftarrow_s \mathcal{R} : \text{LEnc}(pk, b, m; r) = c] \leq \Pr [r \leftarrow_s \mathbb{G} : r \star x = c] \leq 1/|\mathbb{G}| = \text{negl}(\lambda).$$

- ⑥ **CPA Security under Normal Labels:** The proof begins with changing public key to a random one by property ①. Then with overwhelming probability, the label is a lossy one of the random public key. Finally, the property of *random ciphertexts under lossy labels* guarantees the CPA security.
- ⑦ **Ciphertext Randomness in case of Random Messages:** Note that for each component $ct_i = (\mathbf{c}_i, H(\mathbf{c}'_i \oplus m_i))$ of the ciphertext, the first part $\mathbf{c}_i$ is uniformly distributed in $\mathcal{X}$. Moreover, the random message bit m_i is independent of $\mathbf{c}_i, \mathbf{c}'_i$, so $H(\mathbf{c}'_i \oplus m)$ is uniform and independent of $\mathbf{c}_i$. $\square$

Remark 6. When instantiating REGA with CSIDH, any group element $g \in \mathbb{G}$ will be sampled with randomness $r_1 \in \mathcal{R}_1, \dots, r_n \in \mathcal{R}_n$ such that $g = g_1^{r_1} \cdot g_2^{r_2} \cdot \dots \cdot g_n^{r_n}$ and g is described by $g = [r_1, \dots, r_n]$, where $\{g_1, \dots, g_n\}$ is a generating set of $\mathbb{G}$. In fact, we need to use the exponents $r_1, \dots, r_n$ to implement the group action. In $\text{PAKE}_{\text{ga}}^{\text{RO}}$, the hash function H is implemented with $H(pw) := [r_1, \dots, r_n] \in \mathcal{R}_1 \times \dots \times \mathcal{R}_n$. We stress that knowing the exponents $[r_1, \dots, r_n]$ does not lead to any gain to adversary $\mathcal{A}$. For example, ① of LPKE_{ga} holds even if $g = [r_1, \dots, r_n]$ is provided or chosen by adversary $\mathcal{A}$ since $\mathcal{A}$ does not know the representation of t and s and the GA-DDH assumption holds. Similarly, for $\text{PAKE}_{\text{ga}}^{\text{QRO}}$, there is no harm to issue the exponent representation of the labels $\{v_i^0 = [r_{i1}^0, \dots, r_{in}^0], v_i^1 = [r_{i1}^1, \dots, r_{in}^1]\}_{i \in [\lambda]}$ in pp to adversary.

5.3 Instantiations of PAKE

With the instantiations of LPKE and LPKE^+ , we obtain four PAKE schemes in ROM and QROM, namely $\text{PAKE}_{\text{lwe}}^{\text{RO}}, \text{PAKE}_{\text{ga}}^{\text{RO}}, \text{PAKE}_{\text{lwe}}^{\text{QRO}}, \text{PAKE}_{\text{ga}}^{\text{QRO}}$.

Corollary 1. *By plugging the LPKE_{lwe} in Fig. 8 (resp. LPKE_{ga} in Fig. 10) scheme in the generic PAKE construction (cf. Fig. 6), we obtain a specific PAKE scheme $\text{PAKE}_{\text{lwe}}^{\text{RO}}$ (resp. $\text{PAKE}_{\text{ga}}^{\text{RO}}$), which UC-realizes the $\mathcal{F}_{\text{pake}}$ functionality based on the LWE (resp. GA-DDH) assumption in ROM.*

Corollary 2. *By plugging the LPKE^+ scheme $\text{LPKE}'_{\text{lwe}}$ in Fig. 9 (resp. LPKE_{ga} in Fig. 10) in the construction of eLPKE in Fig. 7, we obtain eLPKE from the LWE (resp. GA-DDH) assumption in the QROM. Moreover, by plugging the resulting eLPKE scheme in the generic PAKE construction in Fig. 6, we obtain a specific PAKE scheme $\text{PAKE}_{\text{lwe}}^{\text{QRO}}$ (resp. $\text{PAKE}_{\text{ga}}^{\text{QRO}}$), which UC-realizes the $\mathcal{F}_{\text{pake}}$ functionality based on the LWE (resp. GA-DDH) assumption in QROM.*

Acknowledgements. We would like to thank the reviewers for their valuable comments. This work was partially supported by National Natural Science Foundation of China under Grant 61925207 and Grant 62372292, Guangdong Major Project of Basic and Applied Basic Research (2019B030302008), and the National Key R&D Program of China under Grant 2022YFB2701500.

References

1. https://en.wikipedia.org/wiki/Piling-up_lemma
2. Abdalla, M., Benhamouda, F., Blazy, O., Chevalier, C., Pointcheval, D.: SPHF-friendly non-interactive commitments. In: Sako, K., Sarkar, P. (eds.) ASIACRYPT 2013, Part I. LNCS, vol. 8269, pp. 214–234. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-42033-7_12
3. Abdalla, M., Eisenhofer, T., Kiltz, E., Kunzweiler, S., Riepel, D.: Password-authenticated key exchange from group actions. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part II. LNCS, vol. 13508, pp. 699–728. Springer, Heidelberg (2022). https://doi.org/10.1007/978-3-031-15979-4_24
4. Ajtai, M.: Generating hard instances of lattice problems (extended abstract). In: 28th ACM STOC, pp. 99–108. ACM Press (1996). <https://doi.org/10.1145/237814.237838>
5. Alamati, N., De Feo, L., Montgomery, H., Patranabis, S.: Cryptographic group actions and applications. In: Moriai, S., Wang, H. (eds.) ASIACRYPT 2020, Part II. LNCS, vol. 12492, pp. 411–439. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-64834-3_14
6. Ambainis, A., Hamburg, M., Unruh, D.: Quantum security proofs using semi-classical oracles. In: Boldyreva, A., Micciancio, D. (eds.) CRYPTO 2019, Part II. LNCS, vol. 11693, pp. 269–295. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26951-7_10
7. Azarderakhsh, R., Jao, D., Koziel, B., LeGrow, J.T., Soukharev, V., Taraskin, O.: How not to create an isogeny-based PAKE. In: Conti, M., Zhou, J., Casalichio, E., Spognardi, A. (eds.) ACNS 2020, Part I. LNCS, vol. 12146, pp. 169–186. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-57808-4_9
8. Beguinet, H., Chevalier, C., Pointcheval, D., Ricosset, T., Rossi, M.: Get a CAKE: generic transformations from key encapsulation mechanisms to password authenticated key exchanges. In: Tibouchi, M., Wang, X. (eds.) ACNS 2023, Part II. LNCS, vol. 13906, pp. 516–538. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-33491-7_19
9. Bellare, M., Pointcheval, D., Rogaway, P.: Authenticated key exchange secure against dictionary attacks. In: Preneel, B. (ed.) EUROCRYPT 2000. LNCS, vol. 1807, pp. 139–155. Springer, Heidelberg (2000). https://doi.org/10.1007/3-540-45539-6_11
10. Bellare, M., Merritt, M.: Encrypted key exchange: password-based protocols secure against dictionary attacks. In: 1992 IEEE Symposium on Security and Privacy, pp. 72–84. IEEE Computer Society Press (1992). <https://doi.org/10.1109/RISP.1992.213269>
11. Benhamouda, F., Blazy, O., Ducas, L., Quach, W.: Hash proof systems over lattices revisited. In: Abdalla, M., Dahab, R. (eds.) PKC 2018, Part II. LNCS, vol. 10770, pp. 644–674. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-76581-5_22
12. Beullens, W., Dobson, S., Katsumata, S., Lai, Y.F., Pintore, F.: Group signatures and more from isogenies and lattices: generic, simple, and efficient. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part II. LNCS, vol. 13276, pp. 95–126. Springer, Heidelberg (2022). https://doi.org/10.1007/978-3-031-07085-3_4
13. Boneh, D., Dagdelen, Ö., Fischlin, M., Lehmann, A., Schaffner, C., Zhandry, M.: Random oracles in a quantum world. In: Lee, D.H., Wang, X. (eds.) ASIACRYPT 2011. LNCS, vol. 7073, pp. 41–69. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-25385-0_3

14. Boyle, E., Segev, G., Wichs, D.: Fully leakage-resilient signatures. In: Paterson, K.G. (ed.) EUROCRYPT 2011. LNCS, vol. 6632, pp. 89–108. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-20465-4_7
15. Canetti, R., Dachman-Soled, D., Vaikuntanathan, V., Wee, H.: Efficient password authenticated key exchange via oblivious transfer. In: Fischlin, M., Buchmann, J., Manulis, M. (eds.) PKC 2012. LNCS, vol. 7293, pp. 449–466. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-30057-8_27
16. Canetti, R., Halevi, S., Katz, J., Lindell, Y., MacKenzie, P.: Universally composable password-based key exchange. In: Cramer, R. (ed.) EUROCRYPT 2005. LNCS, vol. 3494, pp. 404–421. Springer, Heidelberg (2005). https://doi.org/10.1007/11426639_24
17. Castryck, W., Lange, T., Martindale, C., Panny, L., Renes, J.: CSIDH: an efficient post-quantum commutative group action. In: Peyrin, T., Galbraith, S. (eds.) ASIACRYPT 2018, Part III. LNCS, vol. 11274, pp. 395–427. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-03332-3_15
18. Coron, J.-S., Patarin, J., Seurin, Y.: The random oracle model and the ideal cipher model are equivalent. In: Wagner, D. (ed.) CRYPTO 2008. LNCS, vol. 5157, pp. 1–20. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-85174-5_1
19. Ding, J., Alsayigh, S., Lancrenon, J., RV, S., Snook, M.: Provably secure password authenticated key exchange based on RLWE for the post-quantum world. In: Handschuh, H. (ed.) CT-RSA 2017. LNCS, vol. 10159, pp. 183–204. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-52153-4_11
20. Don, J., Fehr, S., Majenz, C., Schaffner, C.: Online-extractability in the quantum random-oracle model. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part III. LNCS, vol. 13277, pp. 677–706. Springer, Heidelberg (2022). https://doi.org/10.1007/978-3-031-07082-2_24
21. Gentry, C., Peikert, C., Vaikuntanathan, V.: Trapdoors for hard lattices and new cryptographic constructions. In: Ladner, R.E., Dwork, C. (eds.) 40th ACM STOC, pp. 197–206. ACM Press (2008). <https://doi.org/10.1145/1374376.1374407>
22. Hosoyamada, A., Yasuda, K.: Building quantum-one-way functions from block ciphers: Davies-Meyer and Merkle-Damgård constructions. In: Peyrin, T., Galbraith, S. (eds.) ASIACRYPT 2018, Part II. LNCS, vol. 11272, pp. 275–304. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-03326-2_10
23. Jiang, S., Gong, G., He, J., Nguyen, K., Wang, H.: PAKes: new framework, new techniques and more efficient lattice-based constructions in the standard model. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) PKC 2020, Part I. LNCS, vol. 12110, pp. 396–427. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-45374-9_14
24. Katz, J., Vaikuntanathan, V.: Smooth projective hashing and password-based authenticated key exchange from lattices. In: Matsui, M. (ed.) ASIACRYPT 2009. LNCS, vol. 5912, pp. 636–652. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-10366-7_37
25. Lyu, Y., Liu, S., Han, S.: Universal composable password authenticated key exchange for the post-quantum world. Cryptology ePrint Archive (2024). <https://eprint.iacr.org/2024/374>
26. McQuoid, I., Xu, J.: An efficient strong asymmetric Pake compiler instantiable from group actions. Cryptology ePrint Archive, Paper 2023/1434 (2023). <https://eprint.iacr.org/2023/1434>

27. Micciancio, D., Peikert, C.: Trapdoors for lattices: simpler, tighter, faster, smaller. In: Pointcheval, D., Johansson, T. (eds.) EUROCRYPT 2012. LNCS, vol. 7237, pp. 700–718. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-29011-4_41
28. Peikert, C., Vaikuntanathan, V., Waters, B.: A framework for efficient and composable oblivious transfer. In: Wagner, D. (ed.) CRYPTO 2008. LNCS, vol. 5157, pp. 554–571. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-85174-5_31
29. Quach, W.: UC-secure OT from LWE, revisited. In: Galdi, C., Kolesnikov, V. (eds.) SCN 2020. LNCS, vol. 12238, pp. 192–211. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-57990-6_10
30. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. In: Gabow, H.N., Fagin, R. (eds.) 37th ACM STOC, pp. 84–93. ACM Press (2005). <https://doi.org/10.1145/1060590.1060603>
31. Santos, B.F.D., Gu, Y., Jarecki, S.: Randomized half-ideal cipher on groups with applications to UC (a)PAKE. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part V. LNCS, vol. 14008, pp. 128–156. Springer, Heidelberg (2023). https://doi.org/10.1007/978-3-031-30589-4_5
32. Shoup, V.: Security analysis of *SPAKE2+*. In: Pass, R., Pietrzak, K. (eds.) TCC 2020, Part III. LNCS, vol. 12552, pp. 31–60. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-64381-2_2
33. Zhandry, M.: How to record quantum queries, and applications to quantum indistinguishability. In: Boldyreva, A., Micciancio, D. (eds.) CRYPTO 2019. LNCS, vol. 11693, pp. 239–268. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26951-7_9
34. Zhang, J., Yu, Yu.: Two-round PAKE from approximate SPH and instantiations from lattices. In: Takagi, T., Peyrin, T. (eds.) ASIACRYPT 2017, Part III. LNCS, vol. 10626, pp. 37–67. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-70700-6_2